

UNICONTA

GETTING STARTED WITH UNICONTA DEVELOPMENT



Uniconta A/S

Tryg – The Camp
Klausdalsbrovej 601
2750 Ballerup
Danmark
CVR-nr: 33266928

Hovednummer: (+45) 70 33 16 16

Information: info@uniconta.com

Support: <http://www.uniconta.com/da/supportformular>

Copyright © Uniconta A/S

Mekanisk, fotografisk eller anden form for gengivelse af dette kursusmateriale eller dele deraf, er uden Uniconta A/S's skriftlige samtykke, forbudt ifølge dansk lov om ophavsret.

Table of contents

1. Preface	6
1.1.1. Course requirements.....	6
1.1.2. Schedule	7
2. Introduction	8
2.1. Basic terms and definitions.....	8
2.1.1. Uniconta clients.....	8
2.1.2. NuGet	8
2.1.3. Git.....	8
2.1.4. DevExpress	8
2.1.5. Uniconta development project types	8
2.1.6. Uniconta plugin	8
2.1.7. Uniconta API Key	8
2.1.8. Entities and Controls	9
2.2. Useful Tools	11
2.2.1. Git.....	11
2.2.2. How To: Install Git and use it to get the Uniconta Pages repository.	11
2.2.3. KDiff3	12
2.2.4. Notepad++.....	12
2.2.5. Postman	12
2.3. Getting started with Visual Studio.....	13
2.3.1. Installation.....	13
2.3.2. How To: Create your first Uniconta plugin project	14
2.4. C#	21
2.5. Handling Uniconta updates	21
2.5.1. How to: Update internal plugins and external applications	22
2.5.2. Additional work when updating custom Uniconta pages	22
3. Uniconta API	23
3.1. Uniconta Data Model.....	23
3.2. User defined fields and tables	24
3.2.1. User defined fields & Calculated fields	24
3.2.2. User defined tables	26
3.3. API Basics	27
3.3.1. Get the right Uniconta Nuget API package	27
3.4. Uniconta Plugins	29

3.4.1. How to: Create your first executable plugin	29
3.4.2. How to: Create your first event-based plugin.....	33
3.4.3. How to: Create your first page-based modification.....	35
3.4.4. Notes on creating custom pages.....	39
3.5. User Plugins and Partner plugins	40
3.5.1. How To: Create a partner plugin.....	41
3.6. External integrations.....	42
3.6.1. Integration using a NuGet API for .NET Core	42
3.6.2. OData / REST integration	45
3.7. Working with data and the CrudAPI	46
3.7.1. Insert	46
3.7.2. Query / LoadCache.....	47
3.7.3. Read / UserDocuments	47
3.7.4. Update / StreamingManager	48
3.7.5. Delete	48
4. Tips & Tricks.....	49
4.1. Fields on client and non-client entities.....	49
4.2. Enums and how to parse	49
4.3. Master / Detail queries	50
4.4. Working with Documents and Files	50
4.4.1. Import.....	50
4.4.2. Export	50
4.5. Localization / Multi-language support.....	51
4.6. Performance tips.....	51
4.6.1. Caching.....	51
4.6.2. Use C# functions to handle your data instead of the Uniconta server	52
4.6.3. Sending data back to Uniconta in collections	53
4.7. Get the right prices	53
4.8. Make your Executable plugin run asynchronously	54
4.9. Check for errors	54
4.10. The single DLL file – How to use ILMerge	55
4.10.1. How To: Use ILMerge with a Uniconta plugin:.....	55
4.11. Refresh an overview page in Uniconta client	56
4.12. Generics / Splitlist example	56
4.13. Find the active row when running an event-based plugin	57

4.14. StreamingManager	57
4.15. Create Visual Studio templates.....	58
5. Links	59
6. Exercises.....	60
6.1. Exercise 1: Create Visual Studio Uniconta template	60
6.2. Exercise 2: Event plugin and user-defined fields - Calculating sales prices based on coverage.....	61
6.3. Exercise 3: Executable plugin - Import a CSV file with Inventory items	62
6.4. Exercise 4: Event plugin - Export a CSV file with invoice details when a sales order is invoiced	64
6.5. Exercise 5: External console application – Invoice a list of sales orders from a file	65
6.6. Exercise 6: Modify a Uniconta page to have custom logic	66
6.7. Exercise 7: Event Plugin – Block invoicing	67
6.8. Exercise 8: Executable Plugin – Set field on order.....	67

1. Preface

Welcome to the course "Getting started with Uniconta development".

This course is an introduction to Uniconta development, and will teach you the basics, of how to extend and adjust functionality inside the Uniconta client, as well as connecting to Uniconta in applications outside of the Uniconta client.

This course material can be used as a self study, but it is meant to be used as a reference companion to the live course, which will answer many of the questions that may arise.

Areas covered in this course are listed below:

- Uniconta API keys
- Preparing Visual Studio for Uniconta development
- Git, Nuget, DevExpress
- Basic C#
- Uniconta updates
- Communication between database and client through Unicontas API
- Uniconta entities and structure
- Plugin types and examples
- How to handle User defined tabled and fields
- Import & Export of data to files
- Introduction to Uniconta Pages
- Communicating with Uniconta from external services and programs
- API types - Windows API / .Net Standard / OData API
- Performance optimization
- Minimizing the number of calls to Uniconta
- Tips and tricks on more advanced subjects
- Your own challenges/questions

1.1.1. Course requirements

The following preparations must be done before the start of the course.

- Install Visual Studio Community Edition 2019 or higher, on the Windows 10 machine you will bring to the course. Follow the guide in chapter 2.3.1 to make sure you install the correct components.
- Obtain your personal API-key by signing up via the following link:
<http://www.uniconta.com/become-app-partner/>
The demo key your receive, can be used for both Client and Server integrations.

1.1.2. Schedule

Day 1

- 09:00 - 10:30: Introduction / Basic C# and tools / Exercise
- 10:30 - 12:00: Uniconta API / Data model / Internal plugins
- 12:00 - 12:30: Lunch
- 12:30 - 16:30: Internal plugins / Exercises

Day 2

- 09:00 - 10:00: Internal plugins / Exercises
- 10:00 - 11:30: External integrations / OData-REST API / Exercises
- 11:30 - 12:00: Tips & Tricks
- 12:00 - 12:30: Lunch
- 12:30 - 13:30: Tips & Tricks
- 13:30 - 16:30: Exercises / Q&A

2. Introduction

2.1. Basic terms and definitions

2.1.1. Uniconta clients

Two Uniconta clients are available for installation: The click once client and the Windows store client. The Click-Once client is available as a manual installation, and as an MSI installation, if you need to deploy the client to many machines.

2.1.2. NuGet

The Visual Studio NuGet package manager is used to download and extend Visual Studio functionality in your development projects. Thousands of NuGet packages exist, and different projects can have need for different packages. Uniconta deliver their API as a Nuget package. Depending on which type of project you are working on, you will need only one of the Uniconta Nuget packages.

2.1.3. Git

A free version control system to manage different versions of your code.

2.1.4. DevExpress

DevExpress is a company, who have developed a set of controls, which are used in the Uniconta client. A DevExpress subscription is available for purchase (See the links section). If you are serious about developing Uniconta plugins, or want to use the DevExpress controls in your own projects, then a DevExpress subscription is necessary.

2.1.5. Uniconta development project types

Uniconta Plugin: A Visual Studio Class Library project, which ends up as a single DLL file. This DLL file is created as a plugin inside Uniconta, and works inside the Uniconta client, extending the standard functionality.

External integration or Application: Most commonly a console application or a web service, but many different types of Visual Studio project types are supported. The application can either use a Uniconta NuGet API package, or the OData/REST interface, to communicate with Uniconta.

2.1.6. Uniconta plugin

To use a Visual studio project as a Uniconta plugin, it must be compiled as a single DLL file. This DLL file can be used in two ways. Either as a local plugin, placed in the Uniconta plugin path (Default: `c:\uniconta\pluginPath\`) or as a partner plugin. A partner plugin is the preferred way to distribute your plugin to multiple users. I recommend that you only use a local plugin for testing purposes.

2.1.7. Uniconta API Key

When you build an external integration, or application, which needs to access Uniconta data, AND you want to use one of the Uniconta NuGet API packages for your project, then you must get a unique API key for your project. The API key is needed to identify your application, when it connects to Uniconta and to determine the type of connection you use (Client or Server).

You do not need an API key if you develop an internal Uniconta plugin, and you do not need an API key if you develop an external application, which uses the OData/REST interface.

Two types of API keys are available: Client or Server.

Client API keys are used for solutions, where a single user needs to export or import data from Uniconta, and the application only needs access to data for that user. The existing user credentials are used and the application cannot use multiple users or access multiple companies.

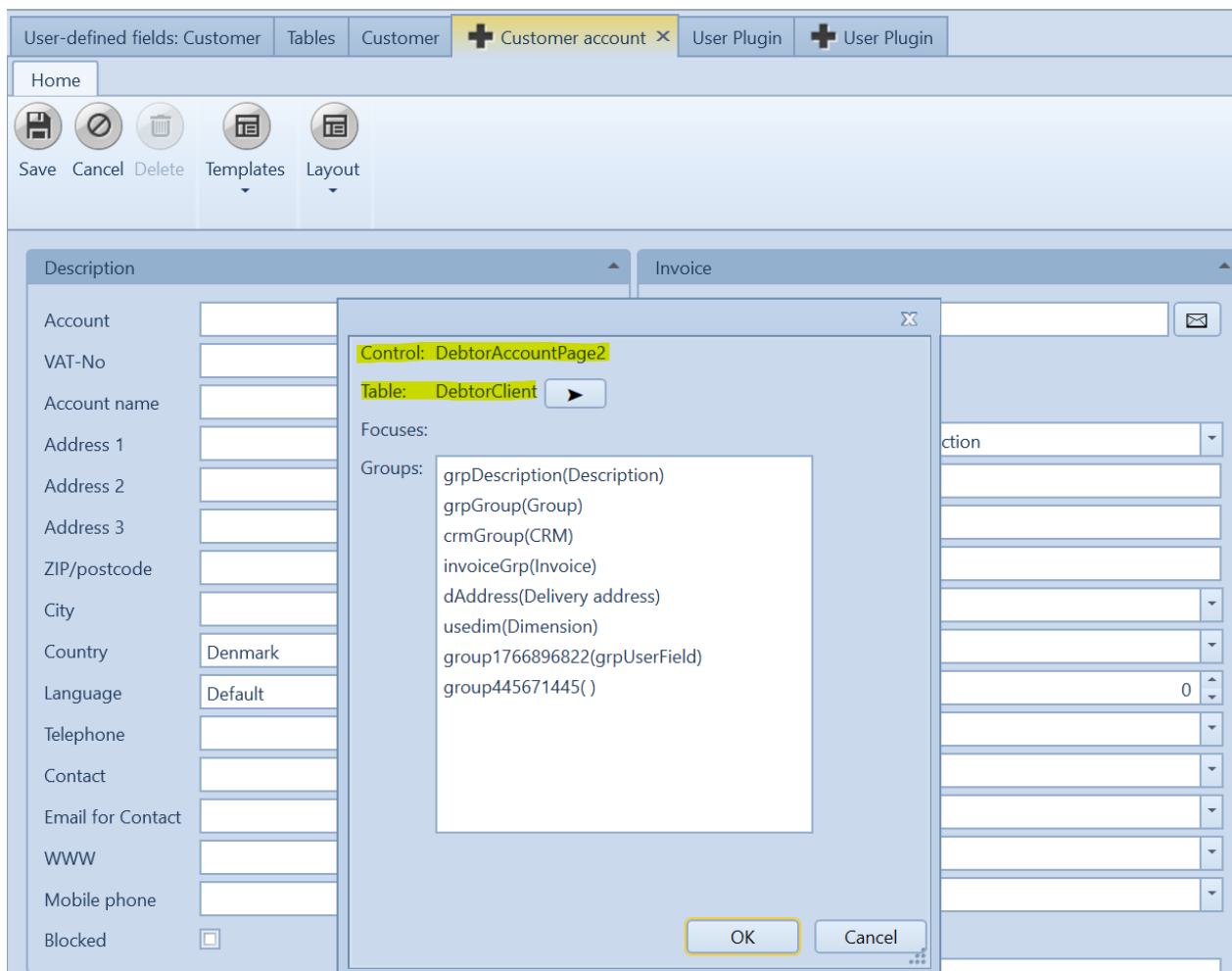
Server API keys are used for solutions, which run on a single machine, and need access to all data inside one, or more companies. A server API application can have access to multiple companies if needed. A special server user is required, and this can be created in the Uniconta client at an extra monthly cost (See <https://www.uniconta.com/da/priser/>). Normal client users cannot be used for server API solutions. One server user per customer is recommended.

2.1.8. Entities and Controls

In this course material, I define all data in Uniconta as entities. An order header is an entity, an order line is an entity, a customer is an entity, and so on.

A Uniconta window is defined as a control. The Customer overview is a control, the single Customer detail view is another control and so on.

You can press F12 on any window in Uniconta to show which Control and Entity (Table) you are currently viewing. You will use these references in your code.



The screenshot shows the Uniconta application interface. At the top, there are tabs for 'User-defined fields: Customer', 'Tables', 'Customer', and 'Customer account'. Below the tabs is a toolbar with icons for 'Save', 'Cancel', 'Delete', 'Templates', and 'Layout'. The main window displays a form for 'Customer account' with fields for 'Account', 'VAT-No', 'Account name', 'Address 1', 'Address 2', 'Address 3', 'ZIP/postcode', 'City', 'Country' (set to 'Denmark'), 'Language' (set to 'Default'), 'Telephone', 'Contact', 'Email for Contact', 'WWW', 'Mobile phone', and 'Blocked'. A modal dialog box is open in the foreground, titled 'Control: DebtorAccountPage2'. It shows 'Table: DebtorClient' and a list of 'Groups' including 'grpDescription(Description)', 'grpGroup(Group)', 'crmGroup(CRM)', 'invoiceGrp(Invoice)', 'dAddress(Delivery address)', 'usedim(Dimension)', 'group1766896822(grpUserField)', and 'group445671445()'. The dialog has 'OK' and 'Cancel' buttons at the bottom.

2.2. Useful Tools

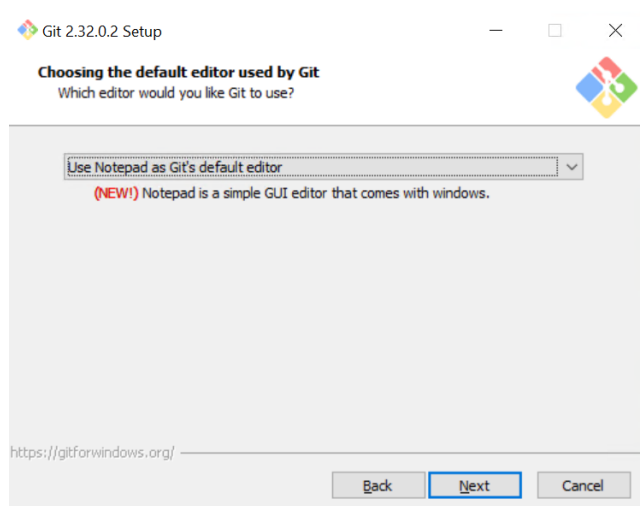
2.2.1. Git

Git is a free code versioning tool. I recommend learning to use it, both if you work in a team or by yourself.

You can download Git for Windows from this link: <https://gitforwindows.org/>

2.2.2. How To: Install Git and use it to get the Uniconta Pages repository.

Accept all the defaults during installation, but I recommend changing the editor to Notepad++ or Notepad, as shown below.

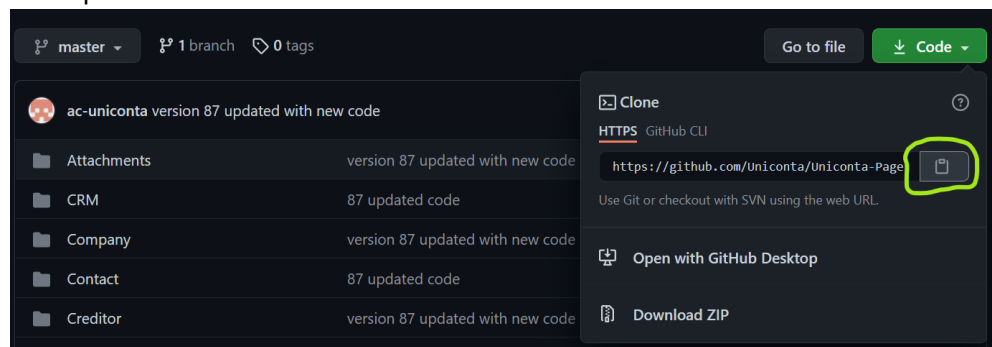


Now that Git is installed, you can right click any folder and choose “Git Bash here”. You are then taken to a Git command prompt, where you can enter your Git commands. Here is a link to a list of the most commonly used commands: <https://www.atlassian.com/git/tutorials/atlassian-git-cheatsheet>

This tutorial will show you how to easily download all Uniconta Pages from Uniconta’s Github.

1. Create a folder, where you want to store the Uniconta Github page files
2. Right click the folder and choose “Git Bash here”
3. Open this link in your browser: <https://github.com/Uniconta/Uniconta-Pages>

Click on the green Code button to unfold it, and then click the marked icon to copy the address to the clipboard



4. Back at the Git command prompt, type “Git clone “ and press SHIFT+INSERT to paste the link into the command window, then finish with space and subfolder name.

The command will be: git clone <https://github.com/Uniconta/Uniconta-Pages.git> v87

This will copy all the files from the Github repository into the v87 folder

Now you can use this folder as a place to copy source files from, when you want to customize any Uniconta pages

If you want to update the folder, to make sure you have the latest files, run the command “git pull” from the same folder with Git Bash.

When version 88 is released, I recommend that you create a new folder for that version.

A demo of how to use Git to handle your own code will be done during the course if time allows.

2.2.3. KDiff3

You may run into a situation where you need to compare the contents of two files. KDiff3 makes it easy to get an overview of differences between files. You can download the tool from this link:

<http://kdiff3.sourceforge.net/>

When it is installed, you can mark the files you want to compare in your file explorer, right click and choose kdiff3 / Compare...

2.2.4. Notepad++

Notepad++ is a text editor with many extra features, compared to Notepad. If you work with a lot of import and export files, it can be helpful. If you open a file with Notepad++ you can easily identify the encoding by using the Encoding menu. Notepad++ can also help you search for specific text in large log files, and across multiple files.

Download link: <https://notepad-plus-plus.org/downloads/>

2.2.5. Postman

If you want to develop solutions based on the Uniconta OData/REST API, or your solutions communicate with other REST API's, then you want to install Postman, so you can test and verify communication.

Download link: <https://www.postman.com/downloads/>

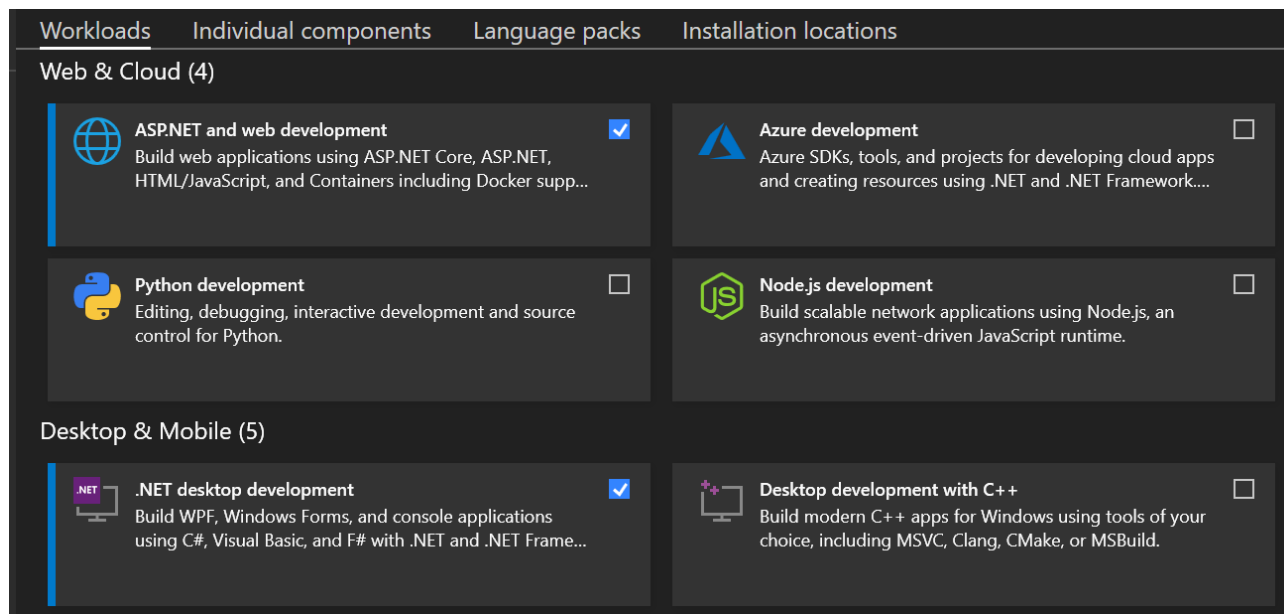
2.3. Getting started with Visual Studio

2.3.1. Installation

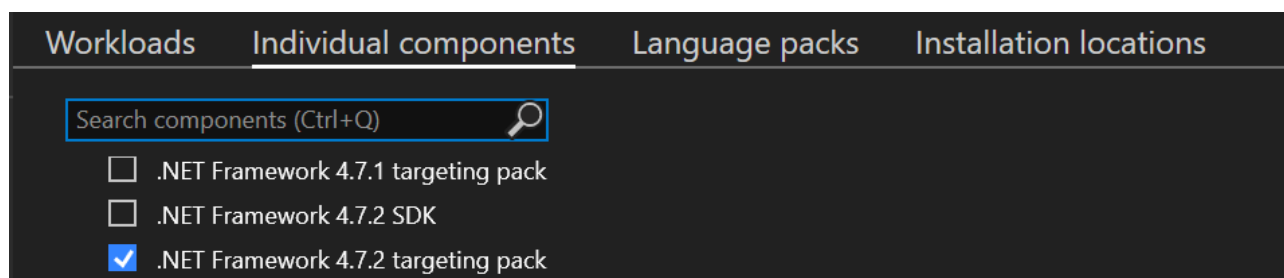
You can download and install the free Visual Studio Community edition from this link:

<https://visualstudio.microsoft.com/vs/community/>

When you install Visual Studio, you must also install “.NET desktop environment” if you want to develop Uniconta plugins, and you must install “ASP.NET and web development” if you want to develop external integrations like web services. I’d recommend always selecting both, as shown below.



Also make sure that you have installed the “.NET Framework 4.7.2 targeting pack”, as shown below. This is needed for Uniconta plugins.

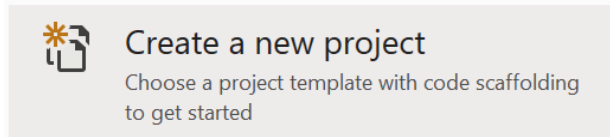


If you have completed the default installation without including these, you can verify and modify the installation by running the Visual Studio Installer and choosing Modify.

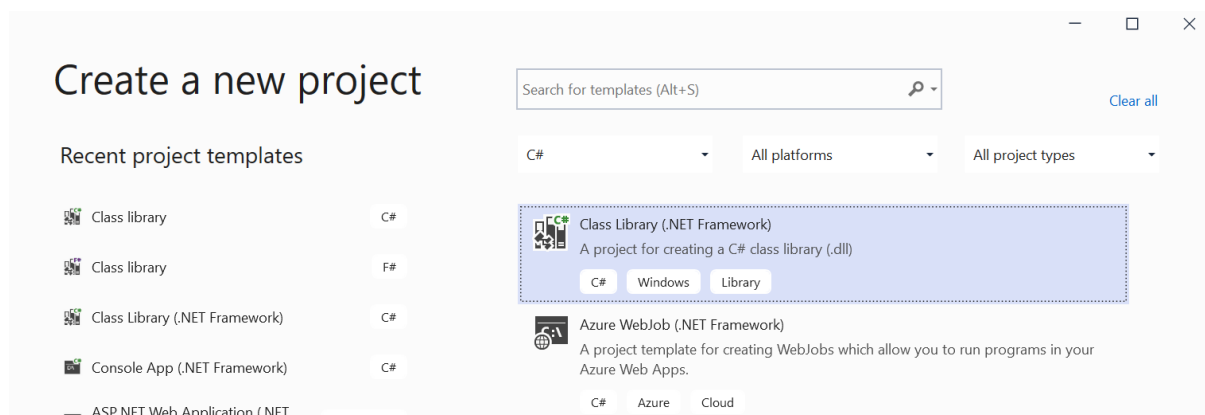
2.3.2. How To: Create your first Uniconta plugin project

This guide will show you how to create an empty Visual Studio project, and configure it, so it is ready to develop a Uniconta plugin.

1. When you start Visual Studio, you will get the opening Wizard. Choose “Create a new Project”:

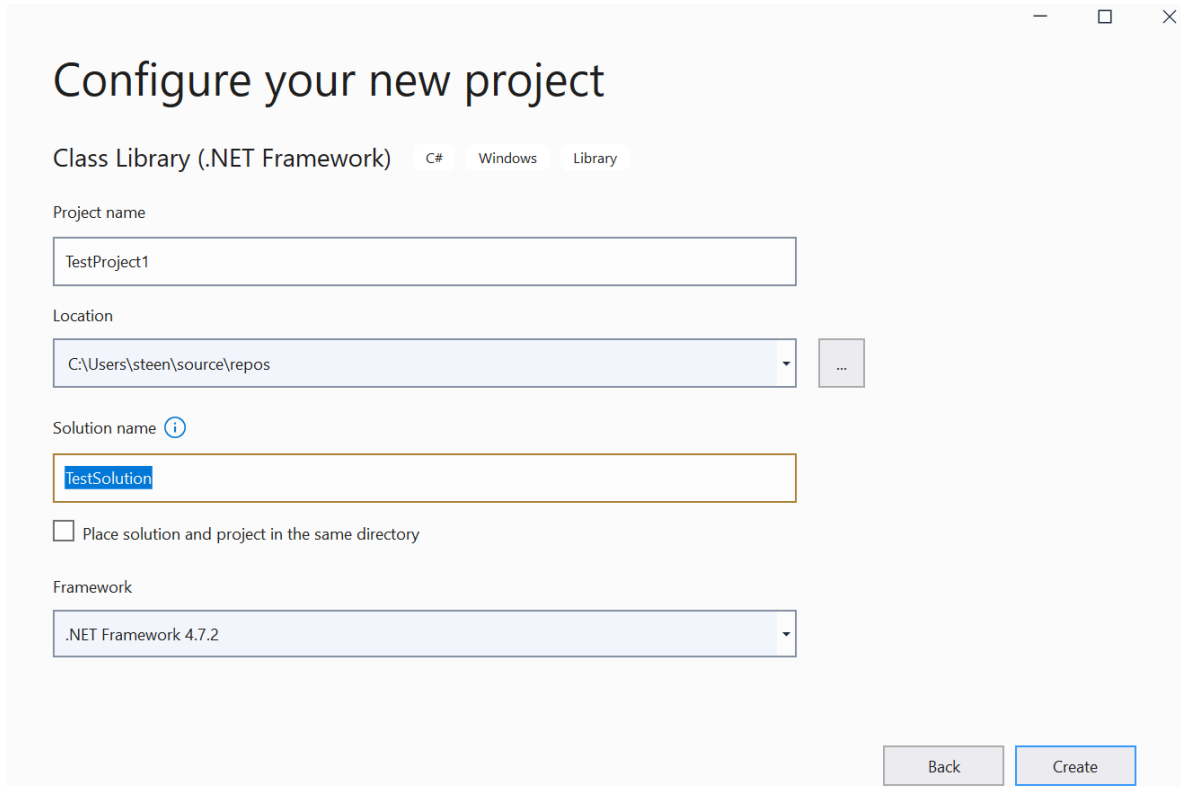


2. Visual Studio will present a list of Project templates. You can filter the list by choosing C# in the Languages dialog and filter further by searching for “class library”. Make sure you select “Class Library (.NET Framework)” for C#, as shown below



For the time being, we are using the Class library for .NET Framework, instead of the one named “Class Library”, which is for .NetStandard. One of the reasons, is that we want to use the WindowsAPI NuGet package for Uniconta plugins, and it uses functions that are not yet part of .Net Standard.

- Fill out the name of your Project and solution and make sure that Framework is set to version 4.7.2. A solution is just a container, and it can hold many projects, so think about how you want to structure your projects. A project per customer in one shared solution might be a good structure, but it depends on many factors. A single project can contain many different Uniconta plugins for the same customer



Configure your new project

Class Library (.NET Framework) C# Windows Library

Project name
TestProject1

Location
C:\Users\steen\source\repos

Solution name ⓘ
TestSolution

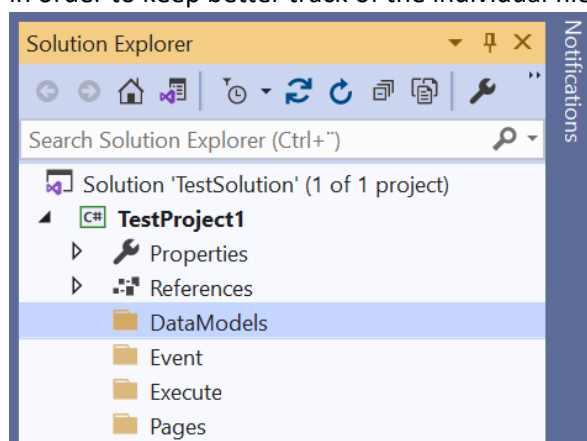
☐ Place solution and project in the same directory

Framework
.NET Framework 4.7.2

Back Create

- Now Visual Studio will create an empty project for you, with a sample class.

I recommend that you create the following folder structure in your Uniconta plugin-based projects, in order to keep better track of the individual files:



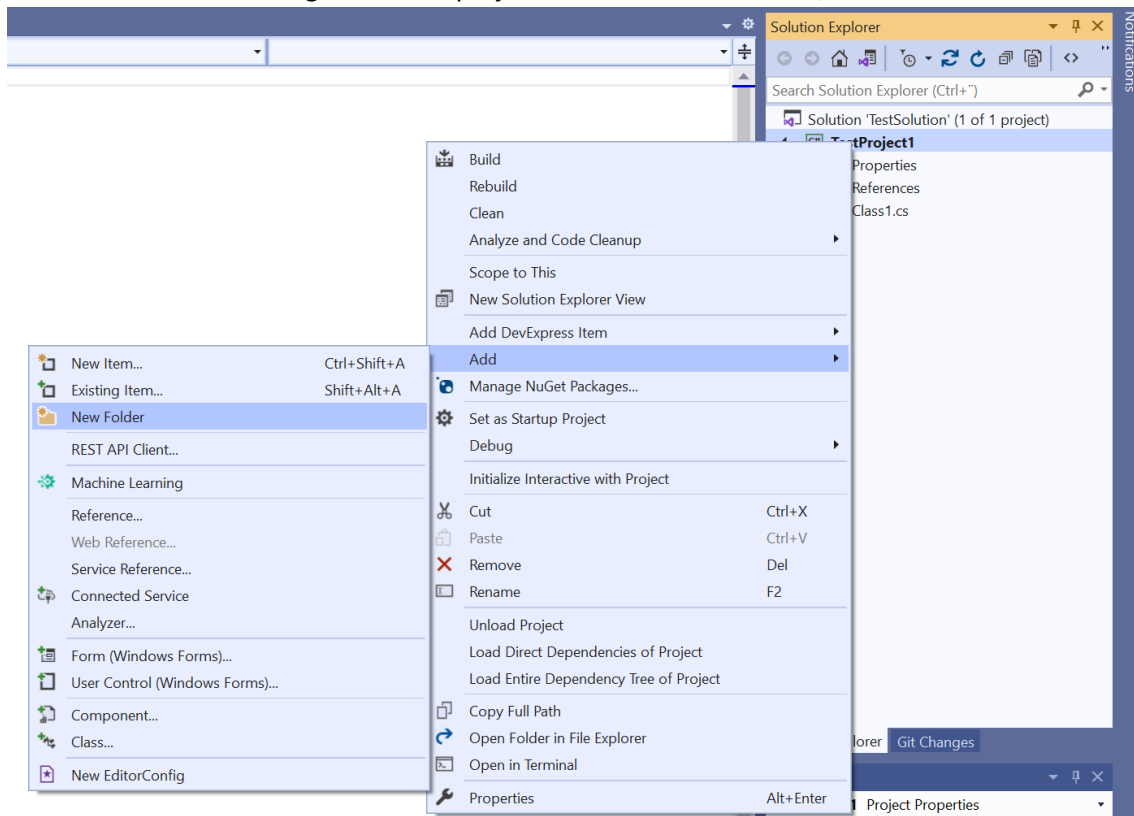
DataModels: Classes that define data objects you need in your project, like user defined tables etc.

Event: Custom classes, which are triggered by an event in the Uniconta client

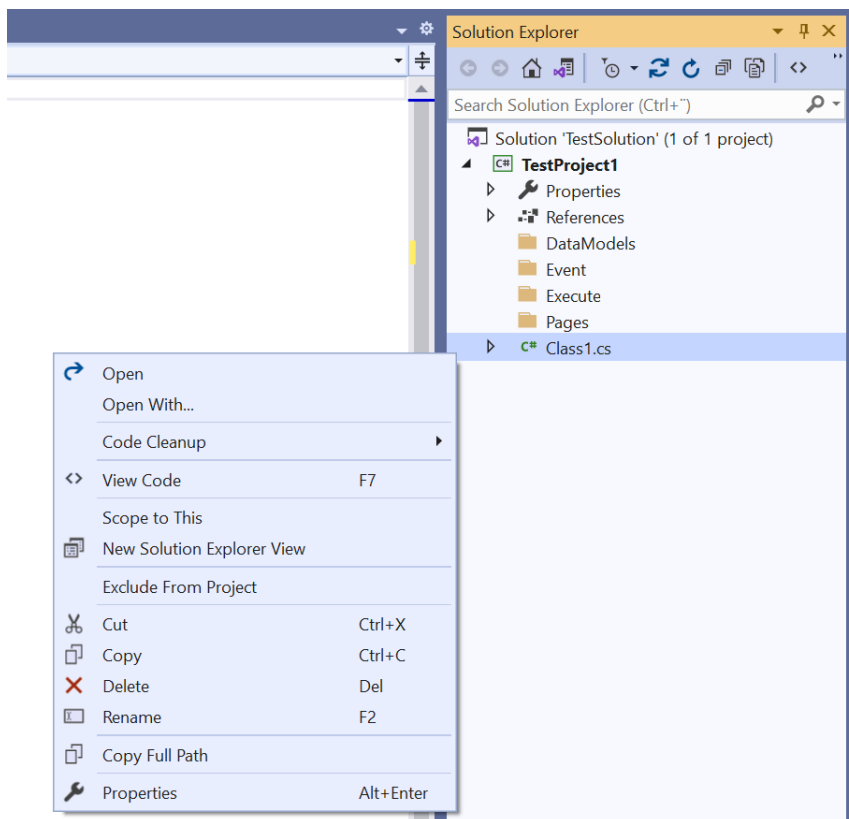
Execute: Custom classes which are triggered when a user runs the plugin from the User Menu

Pages: Holds custom page code, which is taken from Uniconta's Github and modified

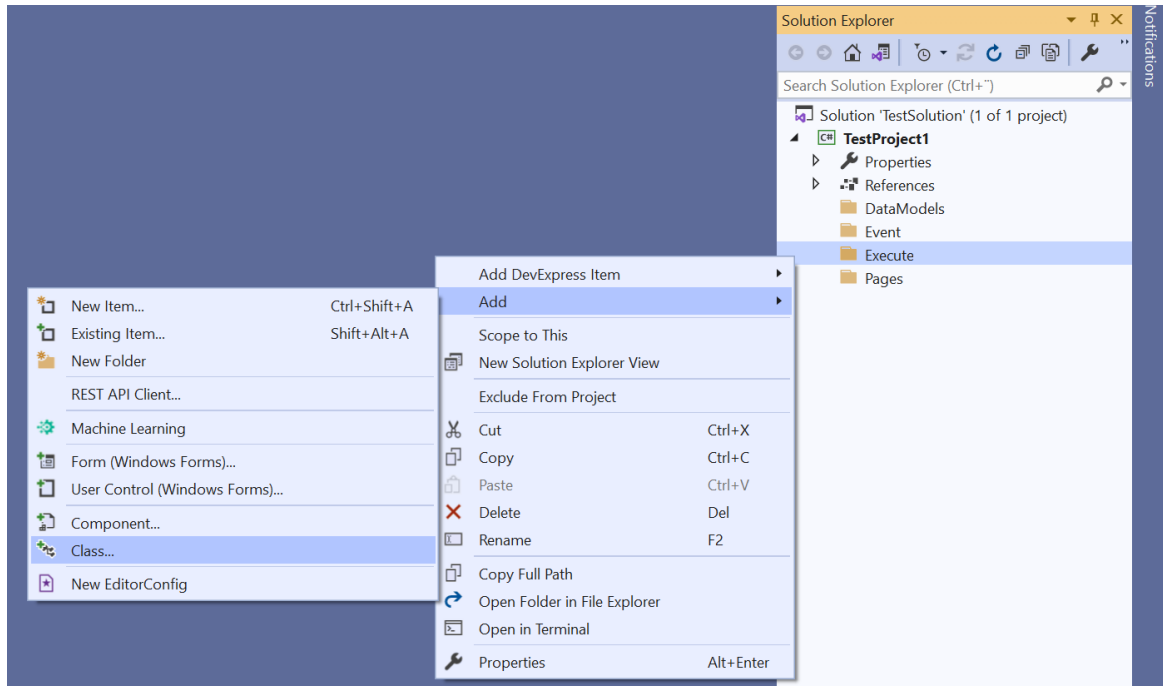
To create a new folder Right click the project name and choose Add / New Folder



5. Delete the default Class1



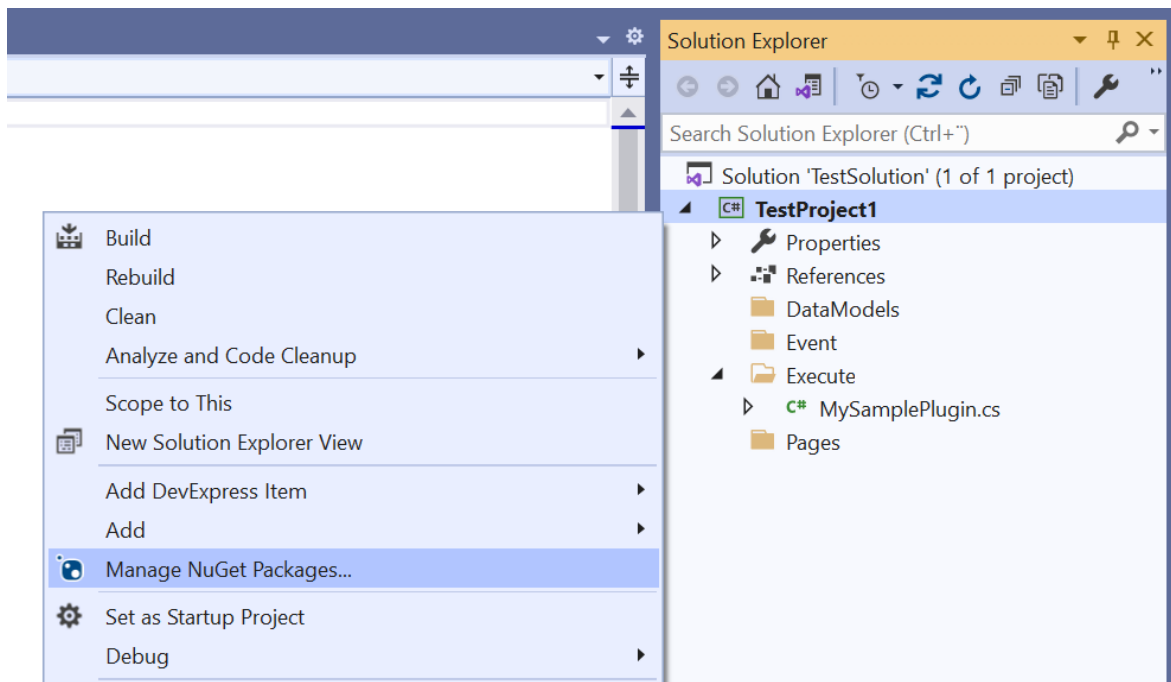
For this sample project, we are creating an executable plugin, so right click the Execute folder and choose Add / Class



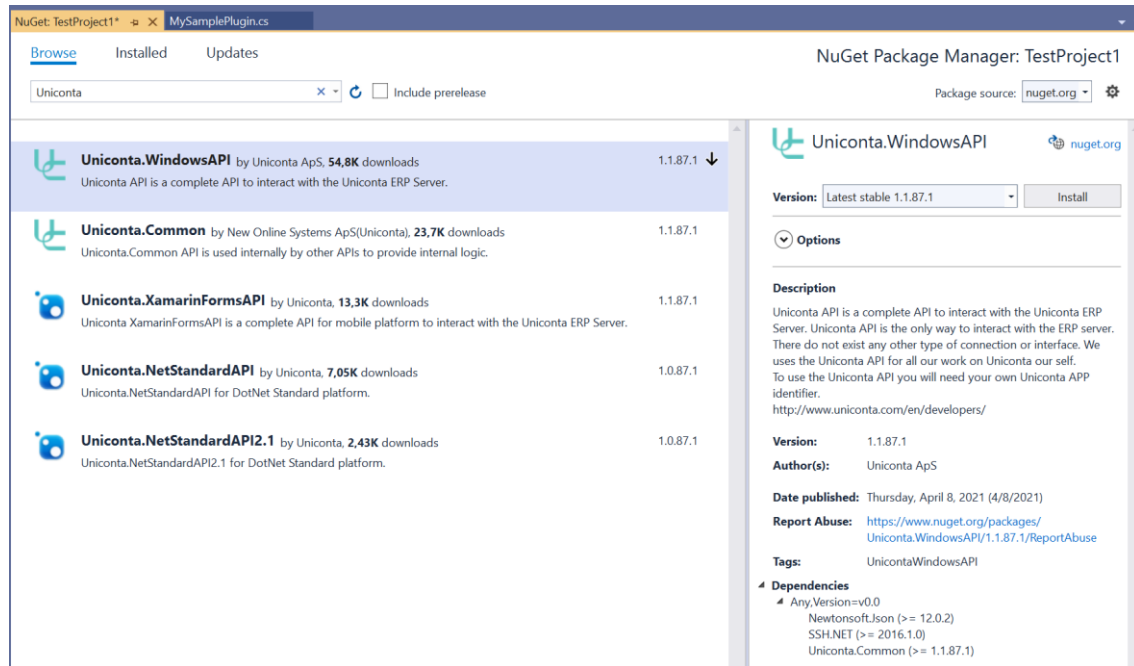
Give the class a name that relates to its function

- Now we need to add a Uniconta NuGet package to our project, and when the package is installed Visual Studio will have learned to handle all of the Uniconta objects we need to communicate with.

Right click the project name and choose “Manage NuGet packages”

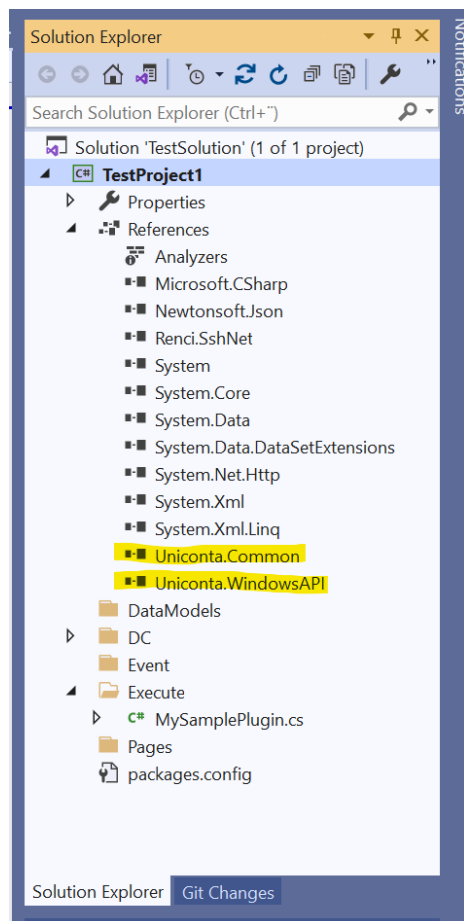


Click on Browse and search for Uniconta. Then click the WindowsAPI as shown below and then click Install:

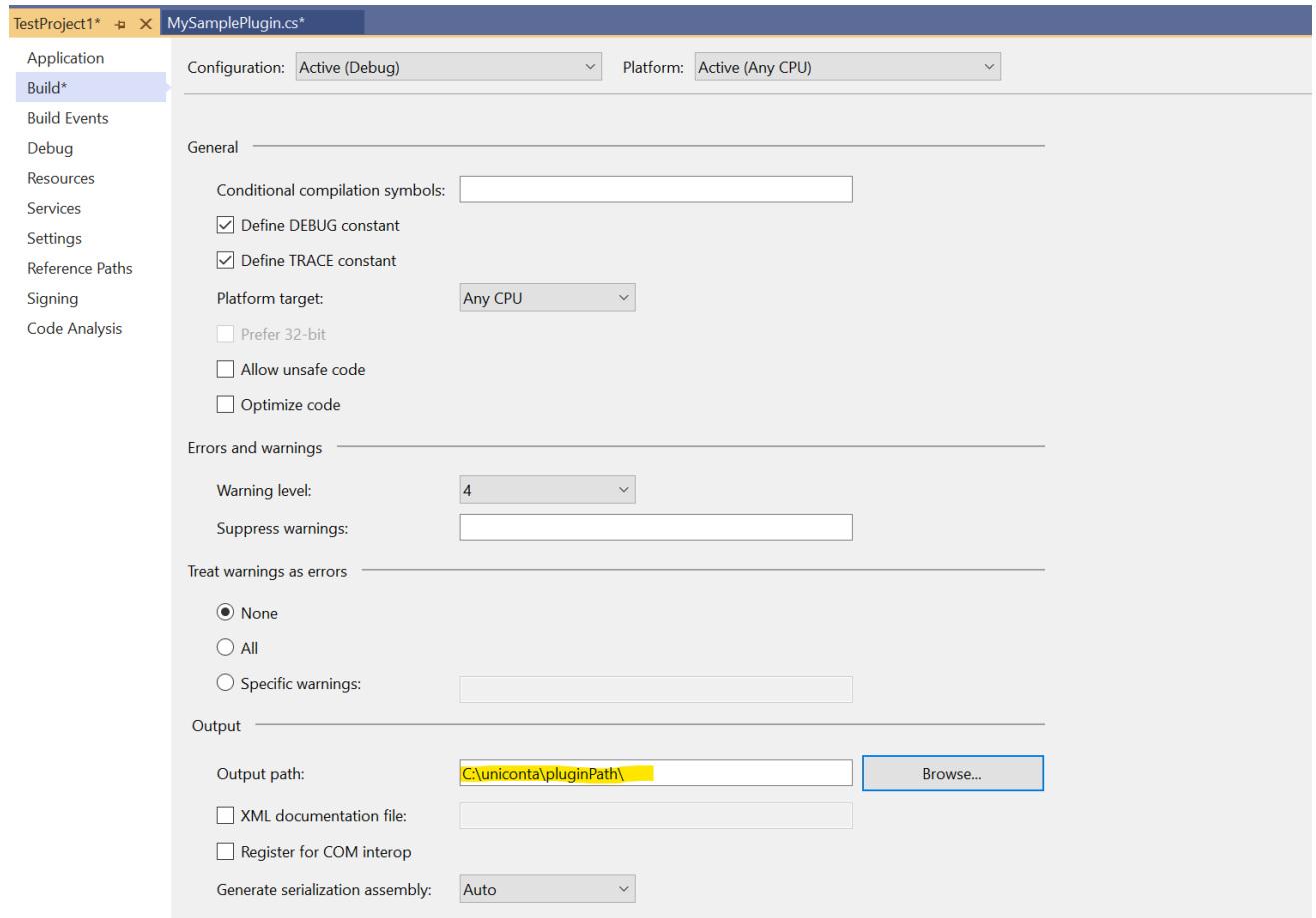


Click OK to the confirmation dialog. A NuGet package can reference many other packages. These are automatically installed.

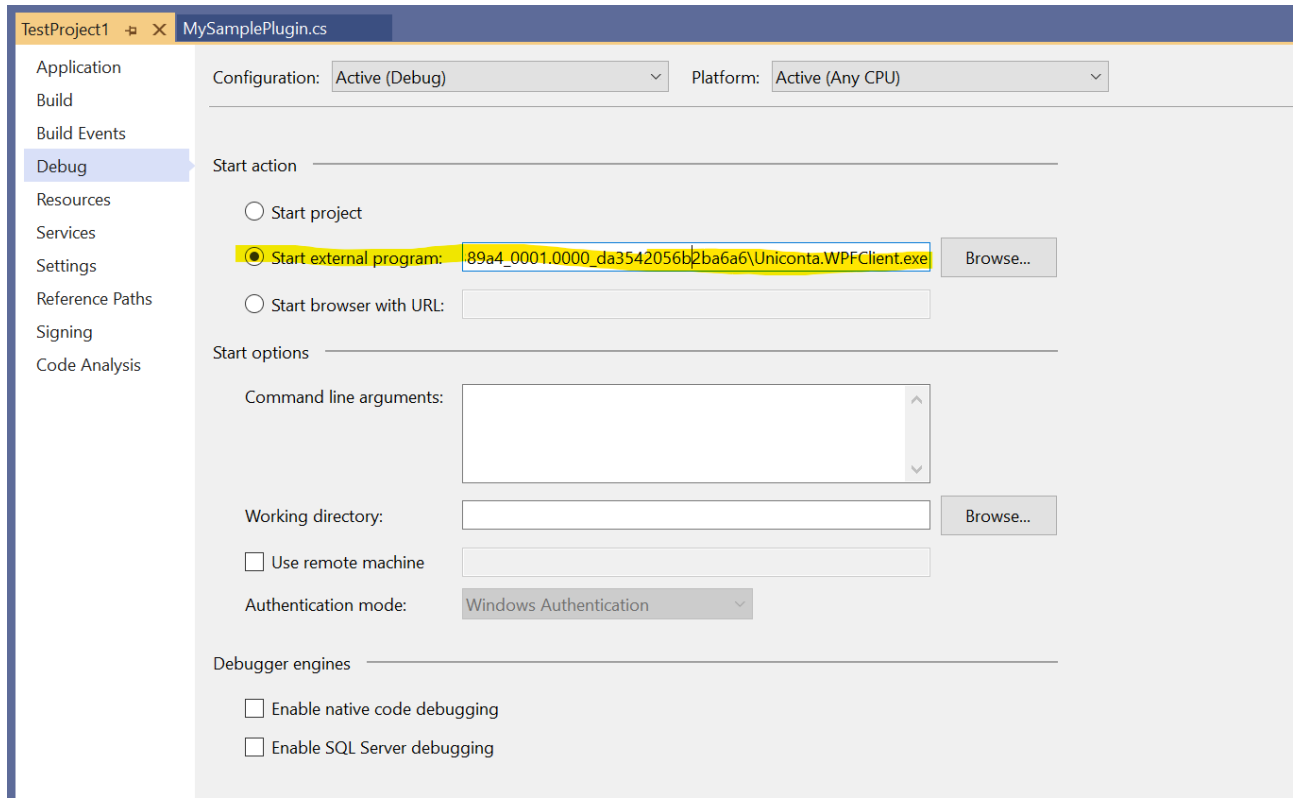
If you fold out References, you will see that Visual Studio now has two Uniconta references:



7. Set the Build path to C:\uniconta\pluginPath. You do this by right clicking the Project name and choose Properties. Then click Build and Browse to the folder or copy and paste it in:



8. In the same window, under the Debug tab, set the location of “Start External Program” to the location of your Uniconta client.



Now Visual Studio knows the Uniconta API, and your project is configured with an output path and debug executable, you can proceed with developing the type of Uniconta plugin that suits your project. You can follow the guide for each type in Chapter 3.

To minimize work, you can create a template project, and use that in the future, instead of going through these steps for each project. Exercise 1 will go through this process.

2.4. C#

C# is the language you will use to develop Uniconta plugins, so a basic knowledge of the language is needed to get started, and the deeper your knowledge, the better your solutions will be.

If you have never programmed in C# before, I can recommend taking a dedicated C# course by Teknologisk (<https://www.teknologisk.dk/kurser/c-sharp-kurser/c513>), or signing up for Pluralsight, which is a huge library of online video courses on many different technologies, including C#.

The link below will take you to a learning path for C# at Pluralsight.

<https://app.pluralsight.com/paths/skill/csharp>

If you sign up with the link below, you can get a 50% discount on the first month or 15% off the first year.

<http://referral.pluralsight.com/mQnfXWq>

A Git beginner course is also available on Pluralsight.

We will go through the basics of C# during the course, and the content will depend on the level of the participants. We will cover Object oriented programming principles, Naming conventions, Reserved keywords, Interfaces, Application scope, Asynchronous versus synchronous programming, Collections, LINQ/Lambda expressions, Generics and several other subjects.

I use these C# naming conventions throughout the course:

<https://www.c-sharpcorner.com/UploadFile/8a67c0/C-Sharp-coding-standards-and-naming-conventions/>

2.5. Handling Uniconta updates

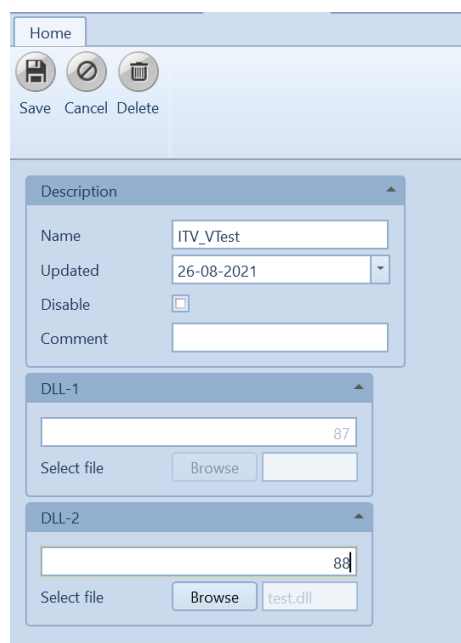
Uniconta release regular updates and fixes to their system, and a few times every year, a major update is released. With every major release, a new version of the NuGet packages are also released. With each new version, it is recommended to also update your project to the new version, and test your projects against the new version of Uniconta.

When a new major version of Uniconta is released, a test version is released about a month prior, and you can use this test version to update and test your projects.

With a Partner plugin, you have the opportunity to prepare and test your plugins with the new version of Uniconta. You can use DLL-2 for your new, updated DLL. If you type in the next version number and save, then Uniconta will automatically use the DLL matching the version of the Uniconta client.

Example for update from version 87 to version 88:

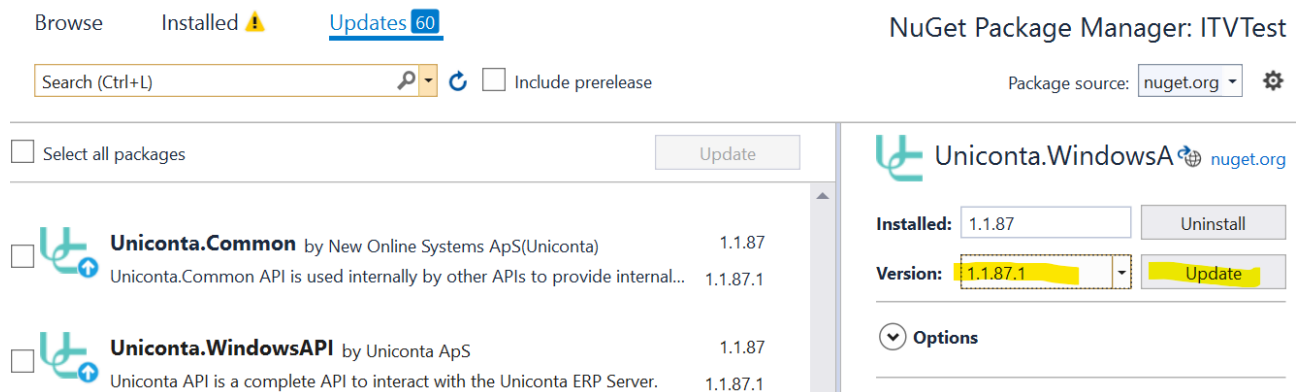
When version 89 arrives, you will then replace DLL-1 and type 89 in that box.



The screenshot shows a software interface with a 'Home' tab and 'Save', 'Cancel', and 'Delete' buttons. Below these are two main sections: 'Description' and 'DLL-1'/'DLL-2'. The 'Description' section contains fields for 'Name' (ITV_VTest), 'Updated' (26-08-2021), 'Disable' (checkbox), and 'Comment'. The 'DLL-1' section has a text box with '87' and a 'Select file' button with a 'Browse' button. The 'DLL-2' section has a text box with '88' and a 'Select file' button with a 'Browse' button and a file named 'test.dll'.

2.5.1. How to: Update internal plugins and external applications

1. Update NuGet Packages (Right click your Project and click “Manage NuGet Packages”)
2. Choose the NuGet Package to update (In this case WindowsAPI) and make sure you have selected the latest version in the Version drop down. Click update and OK to install. Notice that the Uniconta.Common package is automatically updated, as it is referenced by the WindowsAPI package



3. Verify that you can compile the project without errors
4. Verify and test the new DLL file with the test version of uniconta
5. Replace the existing partner plugin DLL file to update immediately, or prepare in advance by using DLL-2

2.5.2. Additional work when updating custom Uniconta pages

If you have a project, where you have imported standard pages from the Uniconta Github, and modified the standard code, then the above process is only the first step.

You will also need wait for the Uniconta Github page to be updated with the new version standard pages, download the new version, import the new pages, and merge your custom code to the new standard code.

The manual steps for this are as follows:

1. Download the new Uniconta standard pages from Github, to a new local repository
2. Backup your custom pages, so you have a copy of your custom code
3. Replace your custom pages with the new standard pages
4. Merge your code from the backed up custom pages to the new standard pages
5. Replace all updated references (ClientTools, UnicontaPages and possibly DevExpress)
6. Test and verify that your custom changes still work.

A more automated process for this can be configured by using Git, different branches and a merge tool.

3. Uniconta API

In order to make it easier to extend Uniconta functionality, and create applications based on Uniconta data, Uniconta has made their API (Application Programming Interface) available to all developers.

With this API you can create any kind of application or extension imaginable. That said, limits of course exist, but the Uniconta API makes Uniconta the most flexible and extensible ERP system, I have experienced.

The Uniconta API is structured in a way that makes it easier to handle. A BaseAPI has a number of child API's, with which, a developer can communicate with Uniconta's internal business logic.

The primary child API we will use in this course is the CrudAPI. CRUD is shorthand for Create, Read, Update and Delete, meaning that by using the methods of this API, you will be able to handle Uniconta data in various ways.

Other interesting child API's exist, but since this course is meant to get you started with Uniconta development, we will not go into details with any of the other child API's, except for this list, where I shortly detail the child API's, which I have used the most, and quickly introduce what they can be used for.

- DebtorCreditor: Can be used to leverage the internal Uniconta logic to get the right prices for order lines, when price lists are involved. This minimizes the custom logic needed when creating purchase- or salesorder lines manually.
- InvoiceAPI: Can be used to create invoices and import old invoices
- UserReportAPI: Can be used to create reports and handle them in your code.
- PostingAPI: Can be used to import physical files to existing vouchers

Several other API's exist. You can explore them through the Object explorer in Visual Studio

3.1. Uniconta Data Model

All data in Uniconta is structured into entities. An entity is a class definition, describing the different properties of the entity, which in itself is an object. All Uniconta data entities are derived from the base class UnicontaBaseEntity.

These entities can be related, as in a debtor is related to a number of orders, and a number of order lines are related to a single order. This is defined as a master relation, where an order always has a debtor as master, and an orderline always has an order as master. There are several advantages of these relations and they will be covered in greater detail later on.

Hint: You can get an overview of all entities in Uniconta by using the Table browser found under Tools / Report Generator. Using the Table browser, you can relate the field label from the Uniconta client, to the field name you have to use in your code.

In code, all Uniconta entities have two representations. A database entity and a client entity. The client entity is derived from the database entity. An example is the Debtor entity, which also exists as DebtorClient. These two entities refer to the exact same data in the Uniconta database. The client entities have several extra fields and are in some cases easier to work with. Chapter 4 has some tips on this.

3.2. User defined fields and tables

In addition to the entities defined by Uniconta, it is possible to extend the data model with your own fields and even your own entities (tables).

User defined fields and tables are covered in another Uniconta course, and will not be covered in great detail here.

3.2.1. User defined fields & Calculated fields

Calculated fields

Recommended naming convention: Begin all calculated field names with cfld.

Calculated fields are essentially small scripts, which are run on other fields and give a result, which is displayed in the calculated field. However, their use is limited to the current entity.

User defined fields

Recommended naming convention: Begin all user defined field names with fld.

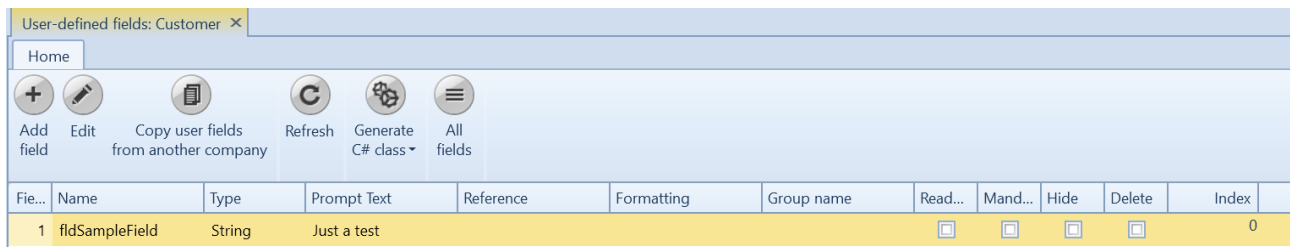
With User defined fields, you can extend Uniconta's entities with your own fields (C# properties), as needed. User defined fields are given a data type and users can then save data in the fields.

When you have extended an existing Uniconta entity with one or more user defined fields, you can access these fields from your code, either by accessing the field index directly, or by name. You can also import the whole extended class into your Visual Studio project. If you do the latter, then you must keep in mind that every time a user defined field is added or changed in Uniconta, you must update your code. The most optimal solution can depend on the number of fields created and other parameters.

Sample User defined field, as created in Uniconta:

The screenshot shows the 'User-defined fields' dialog box for the 'Customer' entity. The dialog has a title bar with two tabs: 'User-defined fields: Customer' and 'User-defined fields: fldSampleField'. Below the title bar is a 'Home' button and three icons: a floppy disk (Save), a circle with a slash (Cancel), and a trash can (Delete). The main area of the dialog is titled 'User-defined fields' and contains a list of fields. The first field is 'fldSampleField' with the following properties:

Property	Value
Name	fldSampleField
Prompt Text	Just a test
Formatting	
Group name	
Type	String
Reference	



Accessing the user defined field from code directly:

```
debtor.GetUserFieldString(0);
```

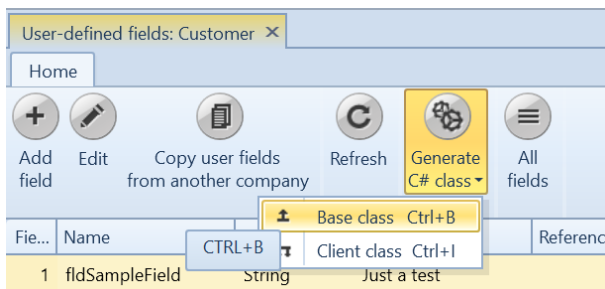
When using this method, you need to use the method which fits with the field type. An advantage is that you always get the value type and you do not have to check for null.

Accessing the user defined field from code by name:

```
string debtor2 = debtor.GetUserField("fldSampleField") == null ? string.Empty :  
debtor.GetUserField("fldSampleField").ToString();
```

When using this way of accessing user defined fields, always remember to check for null, as shown above. You must also do any conversion necessary to work with the different value types in your code. An advantage of this approach is that you only need to know the name of the field.

Importing the extended class from Uniconta:



Choose base class and copy to a new class in Uniconta. Then you can use that class to access all debtor fields by their names, including the user defined fields.

If you have many user defined fields added to a Uniconta entity, it can be easier to work with the fields, if you create this separate class for the entity. If you take this approach, it is important to remember to update your class whenever you make changes to any of the fields, add or remove any fields.

Details on all three options will be shown during the course.

Hint: If you have issues converting entities to your custom class, then the StreamingManager can help. This is described in chapter 4.

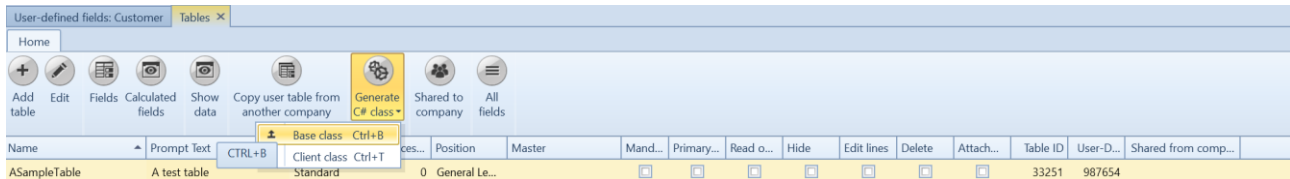
3.2.2. User defined tables

You can create User defined tables inside the Uniconta client, but you can also create them directly in your own code. An example of how to create a user defined table in code is available here:

<https://www.uniconta.com/developers-unipedia-global/api-global/user-defined-tables/>

The link above also describes how the class can be generated programmatically, but I recommend the manual approach to get started.

You can get data from user defined tables by importing its class into your project and then using that class with the CrudAPI. Use the option below and paste the code into a new class in Visual Studio.

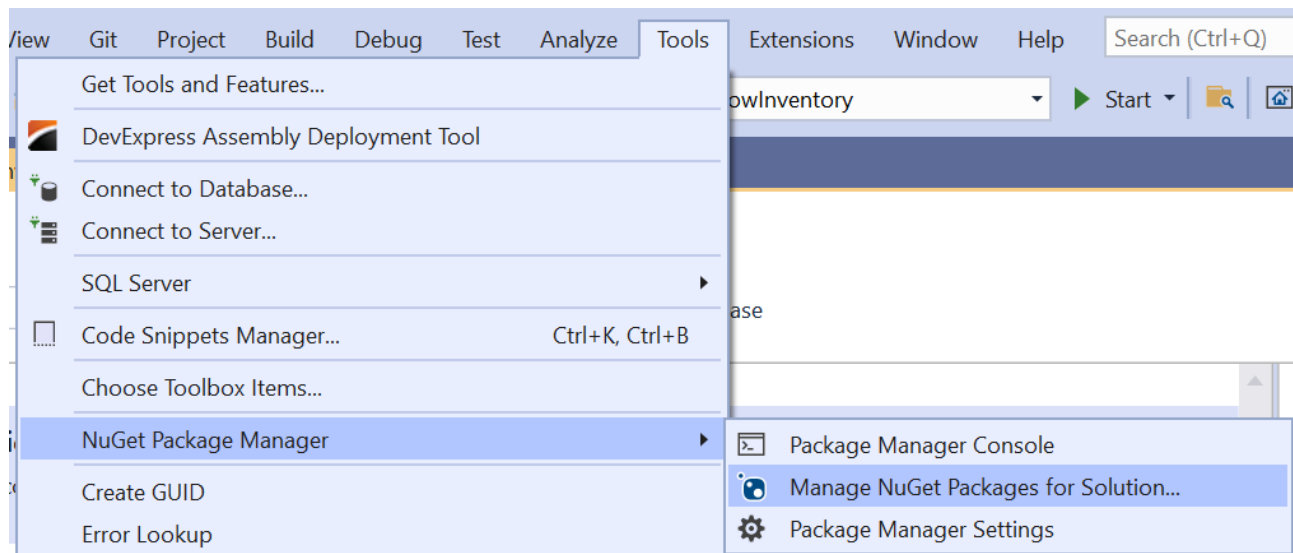


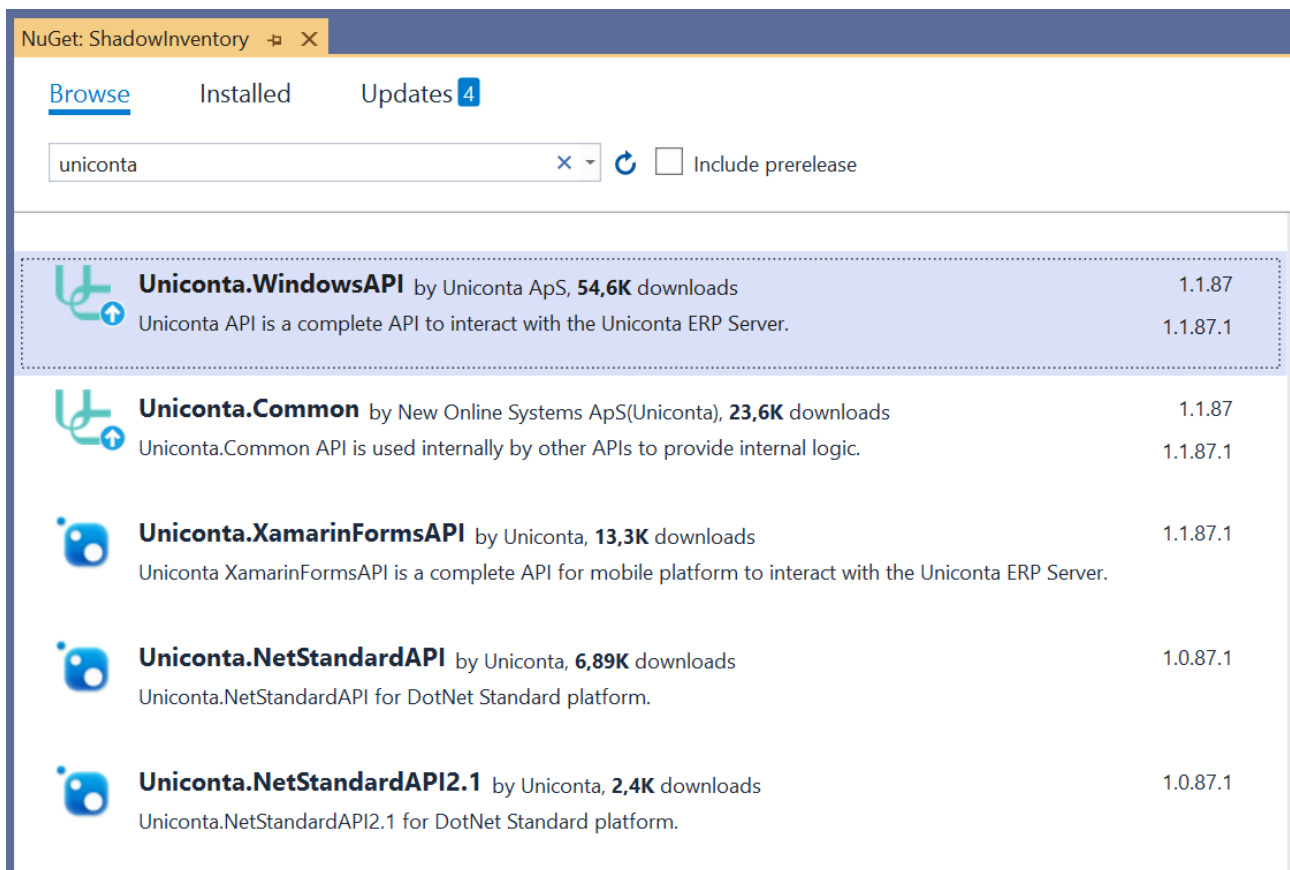
After you have created your custom class, you can work with the use defined table entity in the same way as you can with other Uniconta entities.

3.3. API Basics

3.3.1. Get the right Uniconta Nuget API package

Open the Package Manager as shown below and search for Uniconta





The screenshot shows the NuGet package manager interface with the search term 'uniconta'. The results list several packages, with 'Uniconta.WindowsAPI' highlighted as the first result.

Package Name	Author	Downloads	Version
Uniconta.WindowsAPI	Uniconta ApS	54,6K	1.1.87
Uniconta API is a complete API to interact with the Uniconta ERP Server.			1.1.87.1
Uniconta.Common	New Online Systems ApS(Uniconta)	23,6K	1.1.87
Uniconta.Common API is used internally by other APIs to provide internal logic.			1.1.87.1
Uniconta.XamarinFormsAPI	Uniconta	13,3K	1.1.87.1
Uniconta XamarinFormsAPI is a complete API for mobile platform to interact with the Uniconta ERP Server.			
Uniconta.NetStandardAPI	Uniconta	6,89K	1.0.87.1
Uniconta.NetStandardAPI for DotNet Standard platform.			
Uniconta.NetStandardAPI2.1	Uniconta	2,4K	1.0.87.1
Uniconta.NetStandardAPI2.1 for DotNet Standard platform.			

If you are creating an internal Uniconta Plugin, then you need the Uniconta.WindowsAPI package.

If you are creating an external integration or application, then what you need will depend on the kind of project you are creating.

.Net Core 3.0 or later: Uniconta.NetStandardAPI2.1

.Net Core 2.0 or earlier: Uniconta.NetStandardAPI

A Windows project not targeting .Net Core: Uniconta.WindowsAPI

App for mobile: Uniconta.XamarinFormsAPI

Non-Windows or REST based project: You do not need any NuGet Package to communicate with Uniconta. You can use Uniconta's OData/REST API instead. This interface is limited, when compared to what you can do with the NuGet Packages, but for some solutions, it can be an advantage to not have any dependencies to the Uniconta NuGet Packages.

3.4. Uniconta Plugins

A plugin is defined as a piece of code, which runs inside the Uniconta client. Plugins will not run outside of the Uniconta client.

Three different types of Uniconta plugins exist:

- Executable plugins
- Event-based plugins
- Page modifications

This chapter will explain and show, how to create each type of plugin.

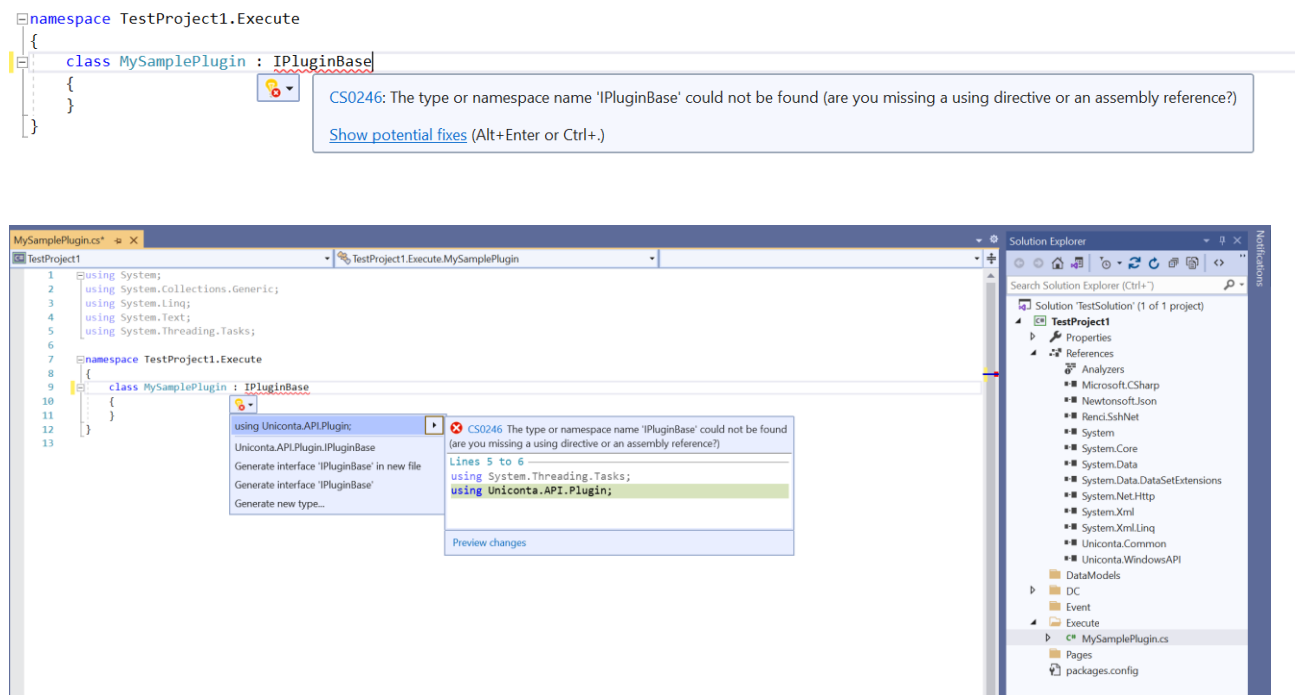
A special type of executable plugin is called a content based plugin. This type of plugin will not be covered in this course.

3.4.1. How to: Create your first executable plugin

1. Create a Visual Studio project, as described in Chapter 2.3
2. Now that Visual Studio knows the Uniconta API, we can begin setting up our sample class to communicate with Uniconta. As this is an executable plugin we need to inherit a specific class from Uniconta, called `IPluginBase`.

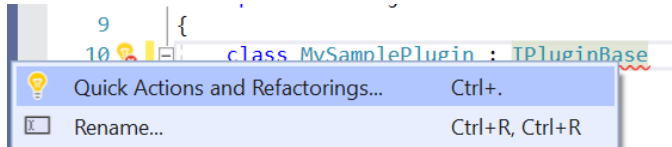
Open your sample class and type “ : `IPluginBase`” after the class name, as shown below. If you don’t have a sample class, then create it

Visual studio will make a red line under `IPluginBase`. If you hover your mouse cursor over the red line, you will get a popup where you can click “Show potential fixes” and then click on the top suggestion “using Uniconta.API.Plugin” (As shown below)

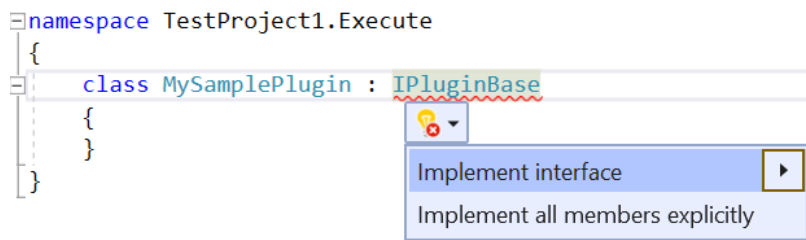


In the future, you can just insert the using statement by hand

3. The red line under IPluginBase is still there. Instead of hovering the mouse cursor over the red line, you can also Right-click on it and choose “Quick Actions and Refactorings” or use the keyboard shortcut CTRL+.



Whichever way you prefer, you must choose “Implement interface”. IPluginBase is defined as an Interface, so we must implement it



- Now we get a lot of code for free, where we need to fill out the blanks. Here is a sample, which will be explained during the course:

```
class MySamplePlugin : IPluginBase
{
    public string ErrorDescription { get; set; }
    public string Name { get { return _name; } }
    private string _name;
    private CrudAPI _crudAPI;
    private ErrorCodes _result;

    public event EventHandler OnExecute;

    public ErrorCodes Execute(UnicontaBaseEntity master, UnicontaBaseEntity currentRow,
        IEnumerable<UnicontaBaseEntity> source, string command, string args)
    {
        //Your custom code goes here
        MessageBox.Show("My first Plugin");

        //The return value tells Uniconta that everything went well
        return ErrorCodes.Succes;
    }

    public string[] GetDependentAssembliesName()
    {
        return null;
    }

    public string GetErrorDescription()
    {
        return ErrorDescription;
    }

    public void Intialize()
    {
        _name = "MySamplePlugin";
    }

    public void SetAPI(BaseAPI api)
    {
        _crudAPI = new CrudAPI(api);
    }

    public void SetMaster(List<UnicontaBaseEntity> masters)
    {
    }
}
```

- Compile the project and make sure there are no errors. We will see that it compiles into a DLL file and if the project is configured, as in the previous guide, the file will be placed in C:\Uniconta\Pluginpath
- The project is now ready for your custom code. The Execute method is called from Uniconta when a user activates the plugin through the user menu, so this is where you will begin to write your custom code.

Notes:

How to understand this line

```
public ErrorCodes Execute(UnicontaBaseEntity master, UnicontaBaseEntity currentRow,  
IEnumerable<UnicontaBaseEntity> source, string command, string args)
```

Master is the owning entity. If you are on a sales order, then its master is a Debtor.

CurrentRow is the line in the overview, which is active.

Source is the complete list in an overview window. If the list is filtered, source will only hold the lines in the active filter

Command is the text you input under Command, when you create the user plugin.

Args is the text you input under Arguments, when you create the user plugin.

Using statements

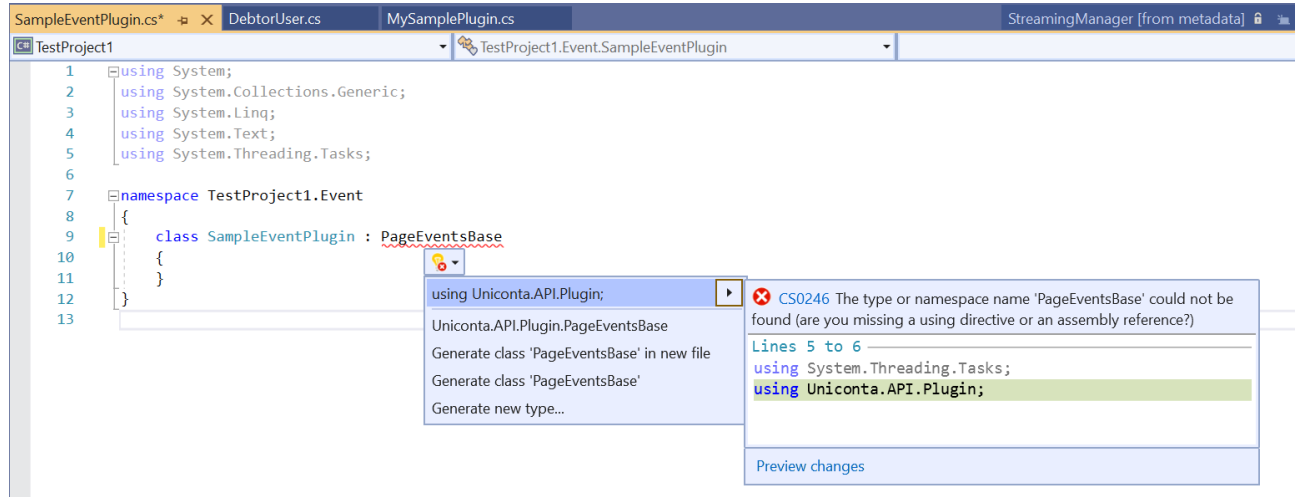
For this plugin to work, you will need the following using statements at the top of your class. These lines tell Visual Studio which other .NET libraries your code uses. Many thousand .Net libraries exist. Some are built-in to the .NET framework, like the first three System references, while others are available as NuGet packages, like the four Uniconta lines, which are all part of the Uniconta NuGet packages

```
using System;  
using System.Collections.Generic;  
using System.Windows;  
using Uniconta.API.Plugin;  
using Uniconta.API.Service;  
using Uniconta.API.System;  
using Uniconta.Common;
```


3.4.2. How to: Create your first event-based plugin

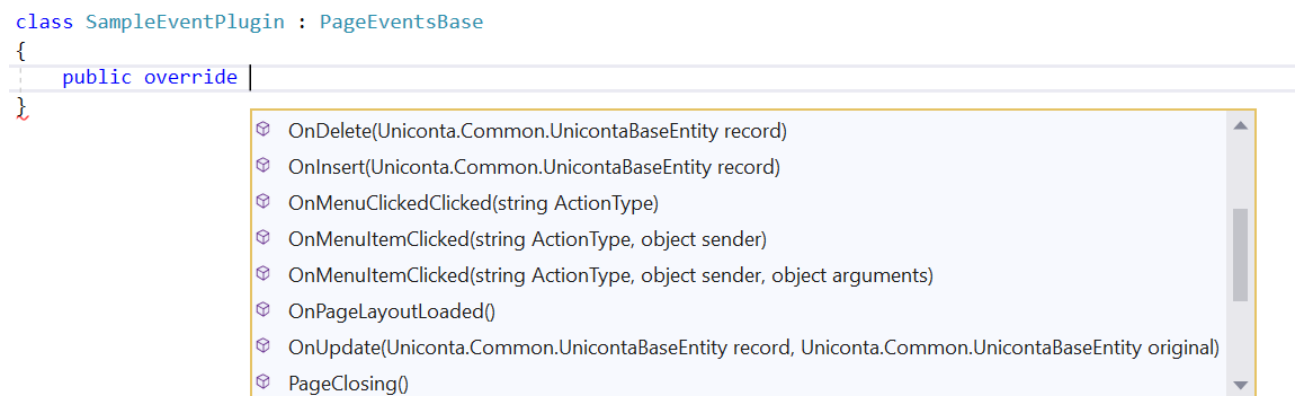
1. Create a Visual Studio project, as described in Chapter 2.3
2. Open your sample class and type “ : PageEventsBase” after the class name, as shown below

Remember to add the necessary using clause, as demonstrated in the executable plugin demo



3. Now that we are inheriting from PageEventsBase, we get a number of functions, which we can override. Not that we are not forced to write any specific methods, we can just override those that match the functionality we wish to achieve.

When you type “public override” and press space you will get a list of the possible methods you can plug into and customize



In this case, we will select OnInsert. We will go through the different possibilities during the course

4. In the sample code below, the code runs every time a Debtor is created and if the Telephone field is not filled out, the record will not be saved

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using Uniconta.API.Plugin;
using Uniconta.ClientTools.DataModel;
using Uniconta.Common;

namespace TestProject1.Event
{
    class SampleEventPlugin : PageEventsBase
    {
        public override ErrorCodes OnInsert(UnicontaBaseEntity record)
        {
            DebtorClient debtor = record as DebtorClient;
            if (debtor.Phone == null)
            {
                ShowMessageBox("You must fill out phone number", "Plugin");
                return ErrorCodes.CouldNotCreate;
            }

            return base.OnInsert(record);
        }
    }
}
```

With an event-based plugin, you are given some objects by default.

api = a CrudAPI object. You can use this in the same way as the CrudAPI object from the executable plugin.

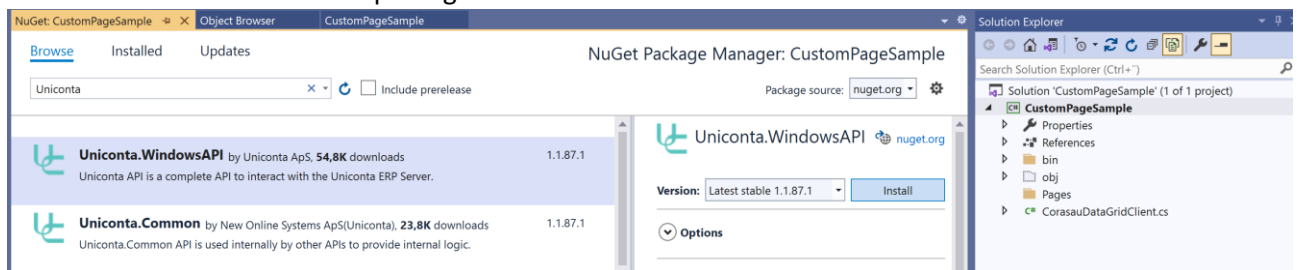
Page = an object representing the active page in Uniconta. You can access this page to get many different kinds of information various ways. For example, to get the current row If you are on an overview page. See the Tips & Tricks chapter for more info on this.

3.4.3. How to: Create your first page-based modification

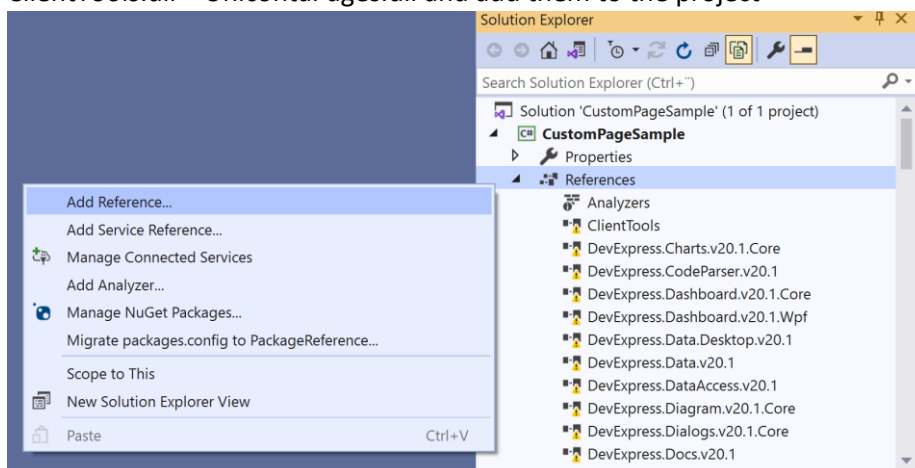
A custom Uniconta Page gives you the opportunity to modify Uniconta standard logic, by importing page files from Uniconta's Github and modifying the screens or logic to your custom need.

Uniconta has created a base project, which can be used as an outset to create page modifications. We will use this project in this sample, to create a modification to the Debtor overview page, where we add a new button to the menu.

1. Follow chapter 2.1 on Git, to download the Uniconta Pages Github repository. Then unpack the zip file CreateCustomPageProject.zip to a new folder in your solution folder
2. Double click CustomPageSample.sln
3. Install the WindowsAPI NuGet package

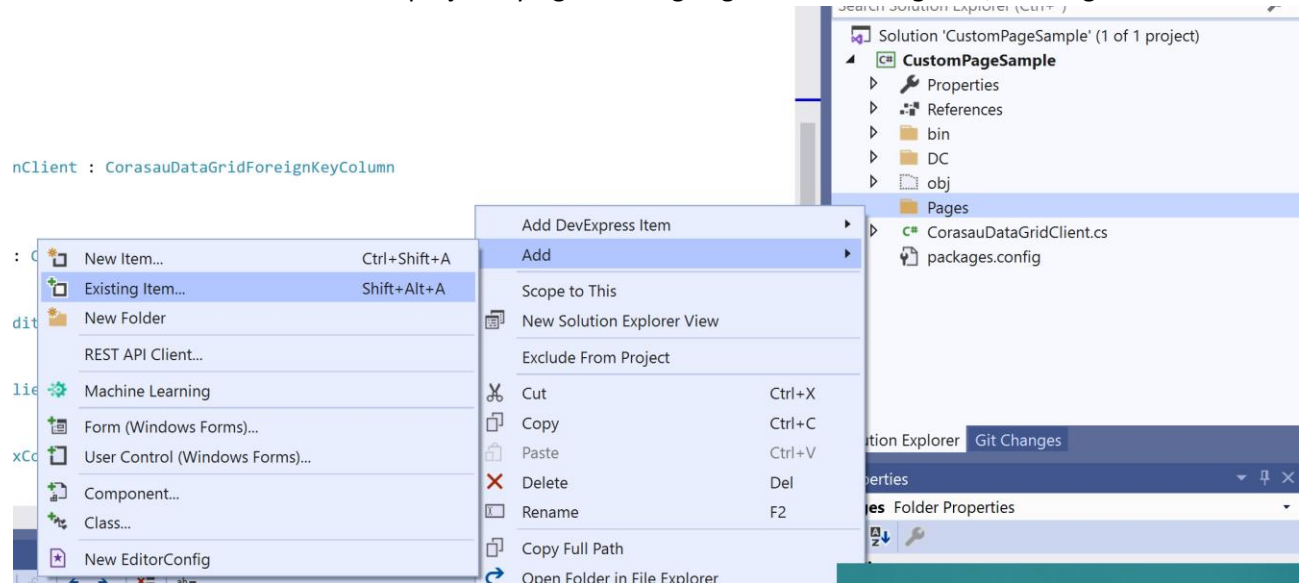


4. Now references with yellow marks need to be updated. Do this by right clicking References and browsing to the Uniconta client folder. Here you mark all dll files that start with DevExpress + ClientTools.dll + UnicontaPages.dll and add them to the project

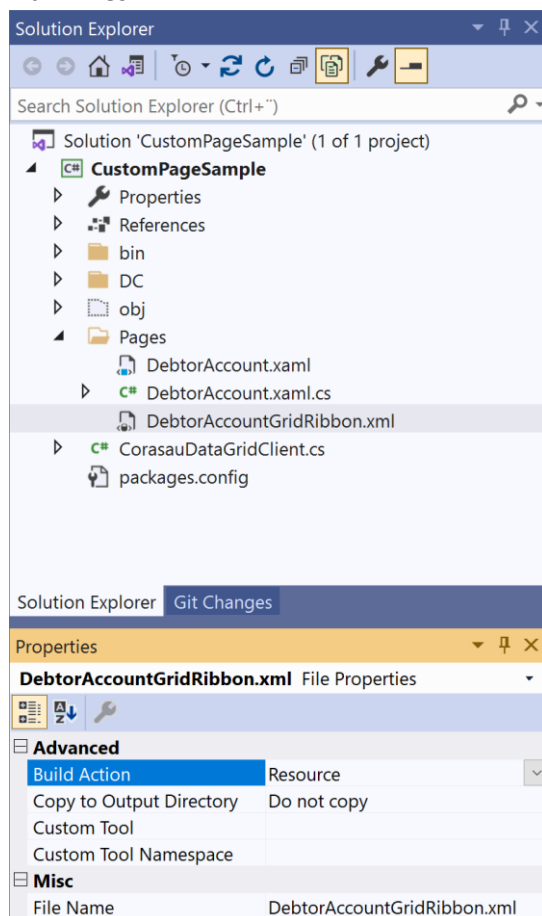


5. When all yellow marks have been cleared, you can import the page files that are needed. Copy the following 3 files from the Debtor subfolder in the Github repository to the Pages folder in your project:
 DebtorAccount.xaml
 DebtorAccount.xaml.cs
 DebtorAccountGridRibbon.xml

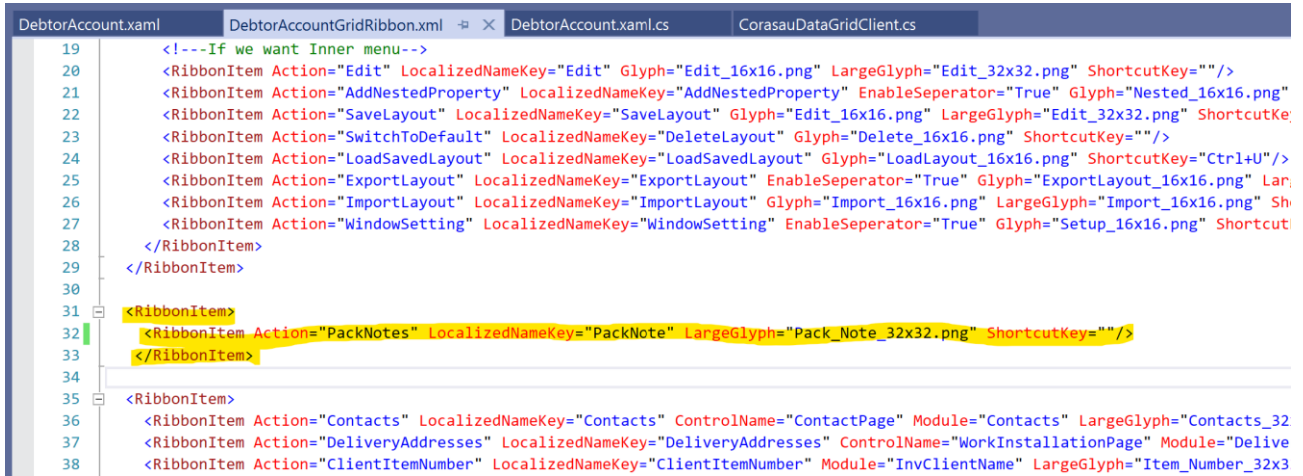
6. Add the files to the Visual Studio project by right clicking Pages and choosing Add / Existing Item:



7. Choose "All files (*.*)" to see all three files, mark them and Click Add
8. Now the files are in the Pages folder in Visual Studio. GridRibbon xml files must have their Build Action set to Resource, as shown below. You do not need to change any properties on the .cs and .xaml files



9. Edit the file DebtorAccountGridRibbon.xml to add a new menu item, with the three new lines shown below

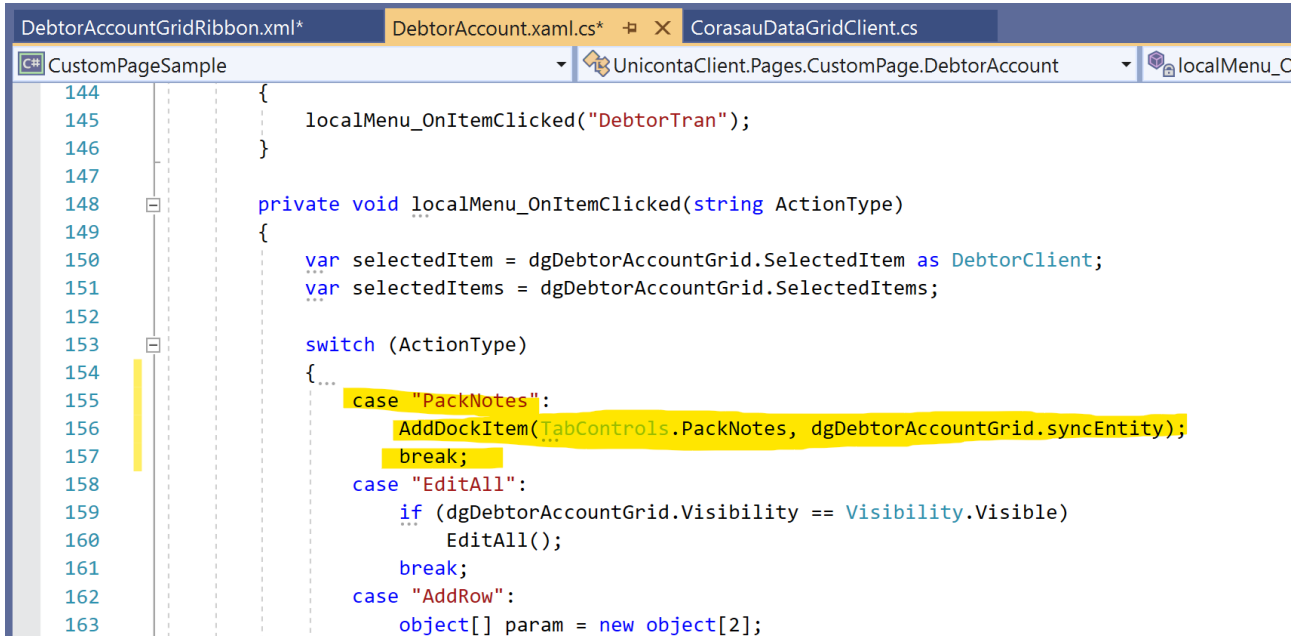


```

19 <!--If we want Inner menu-->
20 <RibbonItem Action="Edit" LocalizedNameKey="Edit" Glyph="Edit_16x16.png" LargeGlyph="Edit_32x32.png" ShortcutKey="" />
21 <RibbonItem Action="AddNestedProperty" LocalizedNameKey="AddNestedProperty" EnableSeparator="True" Glyph="Nested_16x16.png"
22 <RibbonItem Action="SaveLayout" LocalizedNameKey="SaveLayout" Glyph="Edit_16x16.png" LargeGlyph="Edit_32x32.png" ShortcutKey="" />
23 <RibbonItem Action="SwitchToDefault" LocalizedNameKey="DeleteLayout" Glyph="Delete_16x16.png" ShortcutKey="" />
24 <RibbonItem Action="LoadSavedLayout" LocalizedNameKey="LoadSavedLayout" Glyph="LoadLayout_16x16.png" ShortcutKey="Ctrl+U" />
25 <RibbonItem Action="ExportLayout" LocalizedNameKey="ExportLayout" EnableSeparator="True" Glyph="ExportLayout_16x16.png" LargeGlyph="ExportLayout_32x32.png" />
26 <RibbonItem Action="ImportLayout" LocalizedNameKey="ImportLayout" Glyph="Import_16x16.png" LargeGlyph="Import_16x16.png" />
27 <RibbonItem Action="WindowSetting" LocalizedNameKey="WindowSetting" EnableSeparator="True" Glyph="Setup_16x16.png" ShortcutKey="" />
28 </RibbonItem>
29 </RibbonItem>
30
31 <RibbonItem>
32 <RibbonItem Action="PackNotes" LocalizedNameKey="PackNote" LargeGlyph="Pack_Note_32x32.png" ShortcutKey="" />
33 </RibbonItem>
34
35 <RibbonItem>
36 <RibbonItem Action="Contacts" LocalizedNameKey="Contacts" ControlName="ContactPage" Module="Contacts" LargeGlyph="Contacts_32x32.png" />
37 <RibbonItem Action="DeliveryAddresses" LocalizedNameKey="DeliveryAddresses" ControlName="WorkInstallationPage" Module="DeliveryAddresses" />
38 <RibbonItem Action="ClientItemNumber" LocalizedNameKey="ClientItemNumber" Module="InvClientName" LargeGlyph="Item_Number_32x32.png" />

```

10. Edit the file DebtorAccount.xaml.cs and add the lines marked below

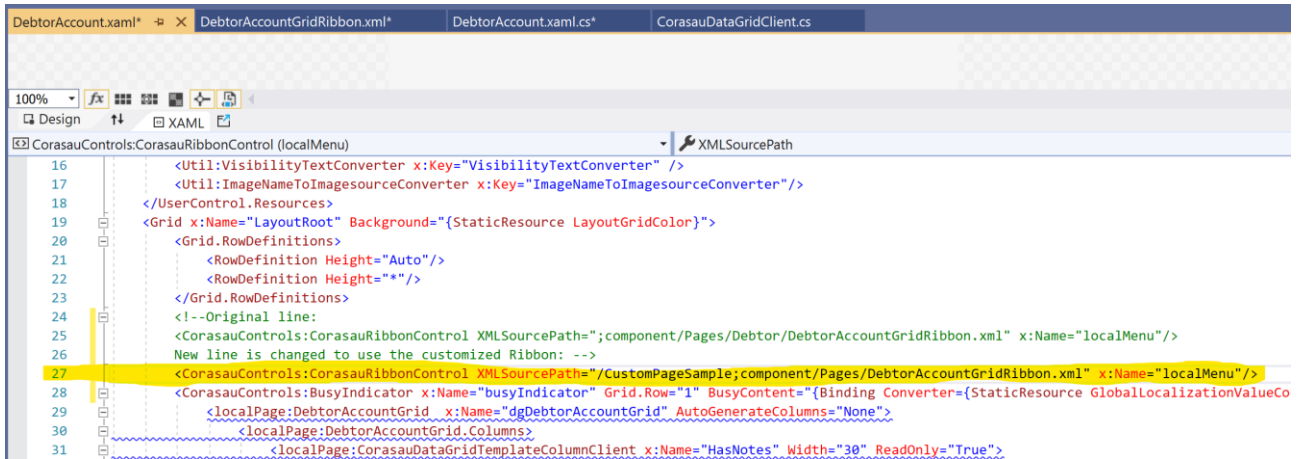


```

144 {
145     localMenu_OnItemClicked("DebtorTran");
146 }
147
148 private void localMenu_OnItemClicked(string ActionType)
149 {
150     var selectedItem = dgDebtorAccountGrid.SelectedItem as DebtorClient;
151     var selectedItems = dgDebtorAccountGrid.SelectedItems;
152
153     switch (ActionType)
154     {
155         case "PackNotes":
156             AddDockItem(TabControls.PackNotes, dgDebtorAccountGrid.syncEntity);
157             break;
158         case "EditAll":
159             if (dgDebtorAccountGrid.Visibility == Visibility.Visible)
160                 EditAll();
161             break;
162         case "AddRow":
163             object[] param = new object[2];

```

11. Edit the file DebtorAccount.xaml and change the line shown below

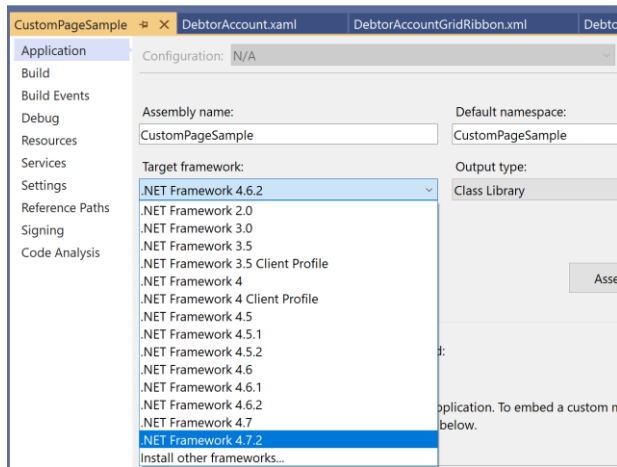


```

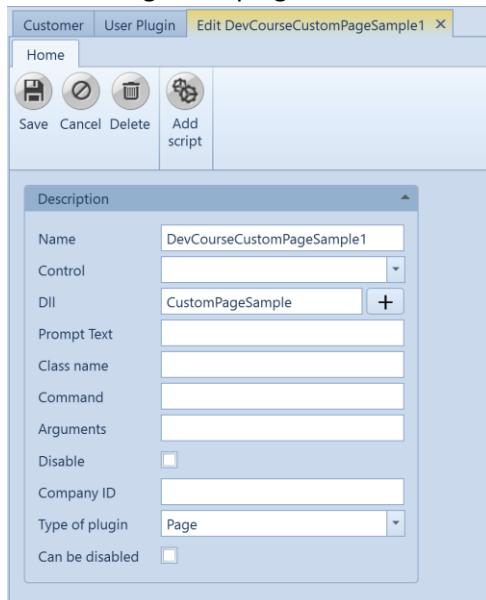
16 <Util:VisibilityTextConverter x:Key="VisibilityTextConverter" />
17 <Util:ImageNameToImageSourceConverter x:Key="ImageNameToImageSourceConverter"/>
18 </UserControl.Resources>
19 <Grid x:Name="LayoutRoot" Background="{StaticResource LayoutGridColor}">
20     <Grid.RowDefinitions>
21         <RowDefinition Height="Auto"/>
22         <RowDefinition Height="*" />
23     </Grid.RowDefinitions>
24     <!--Original line:
25     <CorasauControls:CorasauRibbonControl XMLSourcePath="{component/Pages/Debtor/DebtorAccountGridRibbon.xml" x:Name="localMenu"/>
26     New line is changed to use the customized Ribbon: -->
27     <CorasauControls:CorasauRibbonControl XMLSourcePath="{component/Pages/Debtor/DebtorAccountGridRibbon.xml" x:Name="localMenu"/>
28     <CorasauControls:BusyIndicator x:Name="busyIndicator" Grid.Row="1" BusyContent="{Binding Converter={StaticResource GlobalLocalizationValueCo
29     <localPage:DebtorAccountGrid x:Name="dgDebtorAccountGrid" AutoGenerateColumns="None">
30         <localPage:DebtorAccountGrid.Columns>
31             <localPage:CorasauDataGridTemplateColumnClient x:Name="HasNotes" Width="30" ReadOnly="True">

```

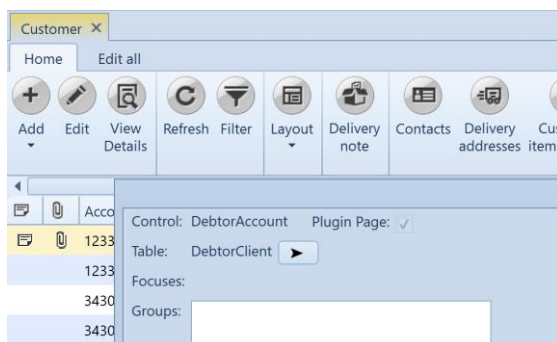
- Now the project can be compiled and added to Uniconta for testing. If you get an error when compiling, then verify Under Properties for the project, that the Target framework is set to version 4.7.2



- Create a Page user plugin in Uniconta as shown below



- Restart Uniconta, and when you open the Customer overview in Uniconta, you should see the new button, and it should lead to the delivery notes for the marked customer. If you press F12 on the Customer overview, you should see that the page is now a plugin page



3.4.4. Notes on creating custom pages

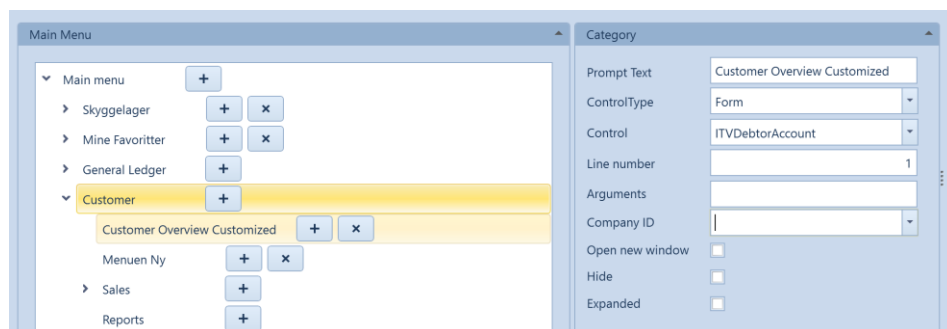
Tag your code when you make changes to Uniconta standard code. It will make it much easier to adjust when new updates are released. A tag could look like this:

```

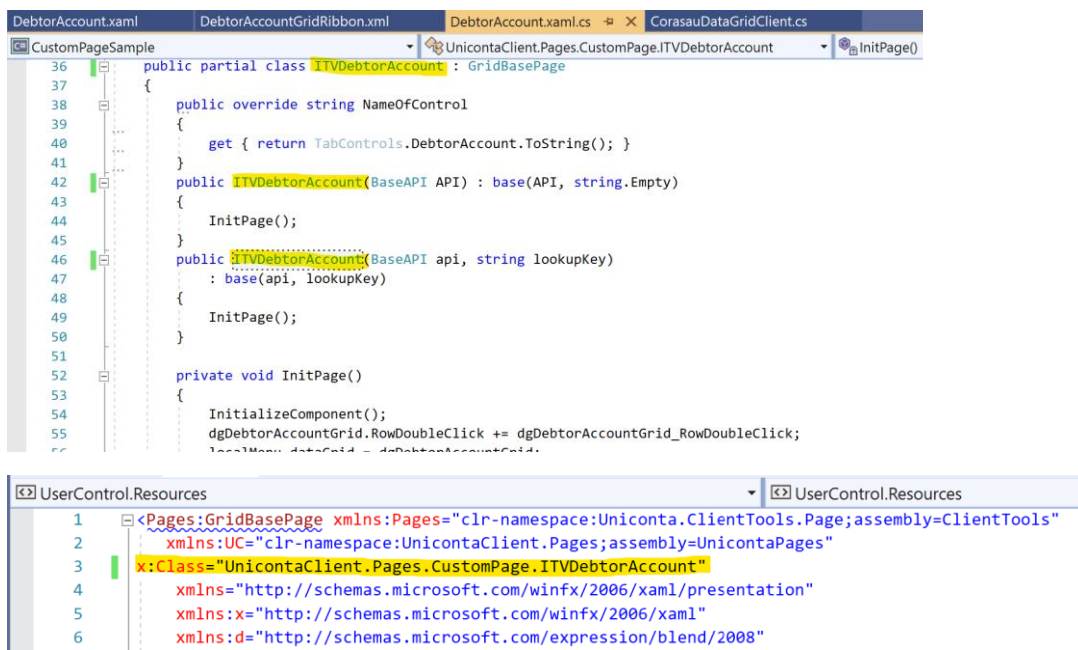
152
153
154      switch (ActionType)
155      {
156          //(+210812 IT Vision (Start))
157          case "PackNotes":
158              AddDockItem(TabControls.PackNotes, dgDebtorAccountGrid.syncEntity);
159              break;
160          //(+210812 IT Vision (Finish))
161          case "EditAll":
162              if (dgDebtorAccountGrid.Visibility == Visibility.Visible)
  
```

Creating custom modifications to Uniconta code should be a last resort. Make sure that there are no other ways to solve the challenge.

If you do not want to replace the standard page, then you can give your page a different name. Then it will show in the controls, when you edit the main menu, and you can have both the standard Uniconta page and your custom page shown.



To get there, you must rename the base class in the .cs file and the .xaml file:



```

36 public partial class ITVDebtorAccount : GridBasePage
37 {
38     public override string NameOfControl
39     {
40         get { return TabControls.DebtorAccount.ToString(); }
41     }
42     public ITVDebtorAccount(BaseAPI API) : base(API, string.Empty)
43     {
44         InitPage();
45     }
46     public ITVDebtorAccount(BaseAPI api, string lookupKey)
47         : base(api, lookupKey)
48     {
49         InitPage();
50     }
51     private void InitPage()
52     {
53         InitializeComponent();
54         dgDebtorAccountGrid.RowDoubleClick += dgDebtorAccountGrid_RowDoubleClick;
55         localMenu.DataContext = dgDebtorAccountGrid;
56     }
57 }
  
```

```

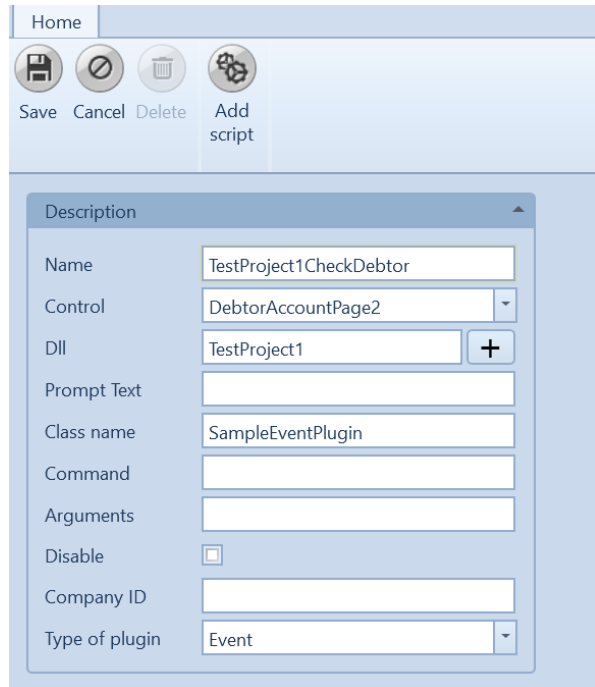
1 <Pages:GridBasePage xmlns:Pages="clr-namespace:Uniconta.ClientTools.Page;assembly=ClientTools"
2   xmlns:UC="clr-namespace:UnicontaClient.Pages;assembly=UnicontaPages"
3   x:Class="UnicontaClient.Pages.CustomPage.ITVDebtorAccount"
4   xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
5   xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
6   xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  
```

If you want to create your own page, I recommend that you use the empty project and an existing page as the base. Delete what you don't need and keep what you can use, and modify it to your need. Then build from there.

3.5. User Plugins and Partner plugins

In order for Uniconta to know and load your projects, you must tell Uniconta some basic details. This is done under Tools / Menus / User Plugin.

Below is shown the sample event plugin described above:



Notes:

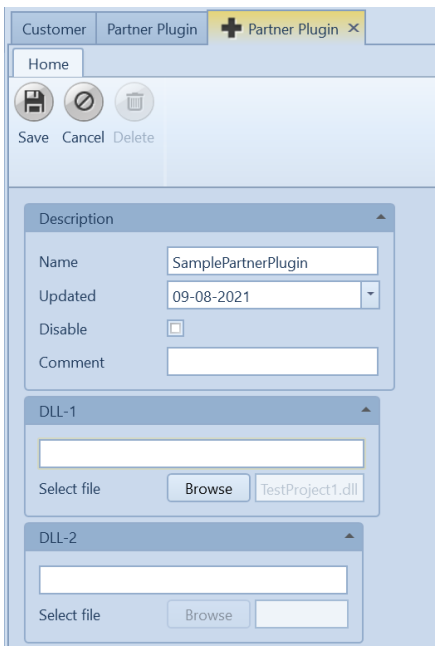
- Name:** Give your plugin a name so you can recognize it later. You can have hundreds of plugins, so it is important to decide on a naming structure and stick to that
- Control:** The name of the Uniconta control, where the plugin is active. Find the control name by pressing F12 on the right screen in Uniconta
- DLL:** Must hold the name of your DLL file or name of your partner plugin. If you type in the name of your DLL file, then this becomes a local plugin, which must be located in the Uniconta pluginpath. If you use the + button, then you will get a list of available partner plugins
- Prompt Text:** Only fill out for Executable plugins. It will be shown in the User menu
- Class name:** The name of your custom class, where your code resides. Copy it from the visual studio project
- Command:** Only fill out if you want to send data to your plugin code. The text here will be available as the command string
- Arguments:** Only fill out if you want to send data to your plugin code. The text here will be available as the args string
- Disable:** Use if your active plugin must be disabled for some reason
- Company ID:** If you fill this out, the plugin will be activated for all users who log into the specific company. If it is not filled out, then the plugin will be active for your user, in all companies

Type of Plugin

- DLL:** An executable plugin, activated through the User Menu
- Event:** An event based plugin, activated when a user triggers the method(s) your code overrides
- Page:** A page based customization, where you have customized existing Uniconta code from Github, or created your own new page from scratch.

3.5.1. How To: Create a partner plugin

To create a partner plugin you go to Administrator / Partner Plugin and fill out the window as described below



- Name:** Give the plugin a unique name to identify it
- Date:** Filled out automatically
- Disable:** Use it to disable an existing partner plugin. It will effectively disable all User plugins based on the partner plugin
- Comment:** Add your own description
- DLL-1:** The box above “Select file” with no label only takes numbers. This box is used to set from which version of Uniconta, the DLL file must be used. Normally you will leave this empty. Next to “Select file” Click Browse and locate the DLL file to save as the partner plugin
- DLL-2:** Uniconta supports two versions of the same partner plugin. Use DLL-2 if you have to support multiple versions and set the numeric version boxes to the correct versions

DLL-2 and the version boxes are used in a scenario, where Uniconta release a new major version of their Uniconta client. In this case, you may want to test and get a new version of your partner plugin ready in advance. If you set the version box to the next version of Uniconta and upload the tested new version of your plugin as DLL-2, it will only be activated when a Uniconta client of the same version logs in. This means that you can prepare for a major update in advance.

3.6. External integrations

External integrations can come in many variations. In this chapter, I will show how to get a connection to Uniconta up and running, in the most basic way possible.

In a real world scenario, I recommend splitting your code into separate classes for Uniconta connection, data manipulation and business logic. This way, you have separate classes for each responsibility.

In this sample, I have put all the code in one class, to make the steps easier to understand.

3.6.1. Integration using a NuGet API for .NET Core

Depending on the type of application you are creating, you will target a specific version of .NET. In the example below, I create a simple .NET Core console application, targeting .Net 5.0

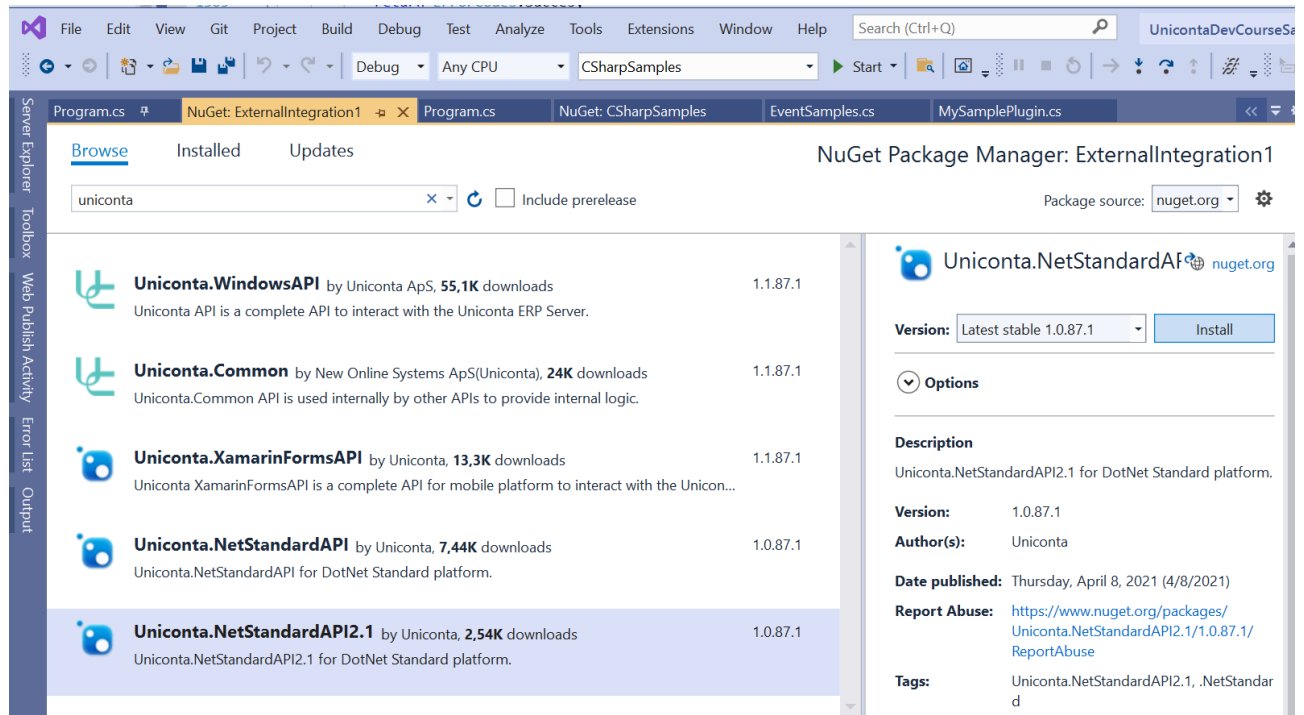
1. Create a new Visual Studio Project – Console Application for .NET Core

The screenshot shows the Visual Studio interface for creating a new project. The top section, 'Add a new project', features a search bar and filters for 'C#' language, 'All platforms', and 'Console' application type. Under 'Recent project templates', 'Uniconta Executable Plugin' and 'Console App (.NET Framework)' are listed. The 'Console Application' template is highlighted, with a description: 'A project for creating a command-line application that can run on .NET Core on Windows, Linux and macOS'. The bottom section, 'Configure your new project', shows the 'Console Application' template selected. The 'Project name' field contains 'ExternalIntegration1'. The 'Location' field is partially obscured by a redacted area.

2. Set the target Framework to .NET 5.0

The screenshot shows the 'Additional information' screen for a 'Console Application'. The 'Target Framework' is set to '.NET 5.0 (Current)'. The 'C#' language and 'Console' application type are selected.

3. Install the NuGet package “Uniconta.NetStandardAPI2.1”



1. Here is the sample code you can copy and past into your class, with comments on each step

```
using System;
using Uniconta.API.Service;
using Uniconta.API.System;
using Uniconta.ClientTools.DataModel;
using Uniconta.Common;
using Uniconta.Common.User;

namespace ExternalIntegration1
{
    class Program
    {
        static void Main(string[] args)
        {
            //Below data should be put in a configuration file and taken from there
            var username = "";
            var password = "";
            var apikey = new Guid("YourAPIKey");
            var companyID = 0;

            //Create the Uniconta connection
            //APITarget must be set to Live. The other settings are used internally by Uniconta
            UnicontaConnection connection = new UnicontaConnection(APITarget.Live);

            //Create a new empty user session
            var session = new Session(connection);

            //Now login to the session
            ErrorCodes result = session.LoginAsync(username, password, LoginType.API, apikey).Result;

            //Double check that the session now has a user
            if (session.User == null || result != ErrorCodes.Succes)
            {
                Console.WriteLine("Error: Unable to login to Uniconta. Error code: " + result);
                Environment.Exit(1);
            }

            //Get a company object for later use
            var company = session.GetCompany(companyID).Result;

            //Verify that the a company is received
            if (company == null)
            {
                Console.WriteLine("Error: Company was not found. Unable to continue.");
                Environment.Exit(1);
            }

            //Now you are logged into Uniconta and can use the API in the same way as you would from a plugin
            var crudAPI = new CrudAPI(session, company);

            //Get all debtors from Uniconta
            var debtors = crudAPI.Query<DebtorClient>().Result;

            //Remember to logout, as we no longer need the Uniconta connection
            session.LogOut();

            //Write each account to the console window
            foreach (var debtor in debtors)
            {
                Console.WriteLine($"{debtor.Account}: {debtor.Name}");
            }

            //Wait for keypress before exiting
            Console.ReadLine();
        }
    }
}
```

3.6.2. OData / REST integration

The OData / REST interface is limited, when compared to what is possible with one of the NuGet API's. For example, you can not create invoices from orders, and there are several other functions that are not possible. However, for simple data extraction and manipulation, the OData/Rest approach, can work just fine.

During the course, we will look at an OData client sample and Postman.

You can download Unicontas sample OData client from here:

<https://www.uniconta.com/download/odata-client/>

3.7. Working with data and the CrudAPI

The CrudAPI is used to manipulate data in Uniconta. The functions demonstrated below can handle single entities, or collections of entities, when Creating, Reading, Updating or Deleting.

As a generalization, it is a good idea to extract all the data you need from Uniconta, and work on that with C# functions, instead of querying entities one at a time, or using filters and other built in Query tools of the CrudAPI.

Also gather what you need to send back to Uniconta in collections, and send back in as few calls as possible. See the performance chapter for more detail on this.

I will demonstrate different options here. These examples can be copied into the sample executable plugin described in chapter. 3.4.1

The samples are included in the Visual Studio project provided to course participants.

3.7.1. Insert

Single Entity

```
//Create a new Customer
DebtorClient newdebtor = new DebtorClient();
//Set the properties you need
newdebtor._Account = "123454321";
newdebtor.Name = "Test Customer";
//Send the customer to Uniconta with the Insert command
_result = _crudAPI.Insert(newdebtor).Result;
//Check result
if (_result != ErrorCodes.Succes)
{
    MessageBox.Show($"Error creating new customer: {_result}");
}
```

Multiple Entities

```
//Create multiple customers
List<DebtorClient> customers2Add = new List<DebtorClient>();

DebtorClient newdebtor1 = new DebtorClient();
newdebtor1._Account = "0001";
newdebtor1._Name = "Test Customer 1";
customers2Add.Add(newdebtor1);

DebtorClient newdebtor2 = new DebtorClient();
newdebtor2._Account = "0002";
newdebtor2._Name = "Test Customer 2";
customers2Add.Add(newdebtor2);

//Send the list to Uniconta with the Insert command
_result = _crudAPI.Insert(customers2Add).Result;
//Check result
if (_result != ErrorCodes.Succes)
{
    MessageBox.Show($"Error creating new customers: {_result}");
    //Investigate the error further
}
}
```

3.7.2. Query / LoadCache

The Query function contacts the Uniconta server with every use, and can either get a complete collection of entities, or it can send a specific query input along, to only get the needed entities.

The LoadCache function works with the existing Uniconta cache, and gets all its data from there. This means that using LoadCache will not generate any calls to the Uniconta server when using a plugin, but instead use the existing data, already cached by the Uniconta client

Query

```
//Get a single Customer by sending a filter query to Uniconta
List<PropValuePair> filter = new List<PropValuePair>();
filter.Add(PropValuePair.GenerateWhereElements("Account", typeof(string), "0001"));
var debtor = _crudAPI.Query<DebtorClient>(filter).Result;

//Get all Customers from Uniconta and find one of the with C# command
List<DebtorClient> debtors = _crudAPI.Query<DebtorClient>().Result.ToList();
var debtor = debtors.Where(x => x.Account == "0001").FirstOrDefault();
```

LoadCache

```
//Using the cache to find a customer (No calls are sent to Uniconta)
DebtorClient debtor = _crudAPI.LoadCache<DebtorClient>().Result.Get("0001");

//Using the cache to get a list of all debtors (No calls are sent to Uniconta)
List<DebtorClient> debtors = _crudAPI.LoadCache<DebtorClient>().Result.ToList();
DebtorClient debtor = debtors.Where(x => x.Account == "0001").FirstOrDefault();
//If no matching debtor was found, the value of debtor is now null
```

Always check for null

```
//Before moving on from each of the above examples, verify that debtor is not null
if (debtor != null)
{
    //Continue the code
}
```

An Alternative

```
//Alternative way to get a specific debtor
DebtorClient debtor = new DebtorClient { Account = "0001" };
_result = _crudAPI.Read(debtor).Result;
//Now the debtor object has all values from the Uniconta database
```

3.7.3. Read / UserDocuments

You use the Read function to read extended data on entities. An example is

```
UserDocsClient[] docs = _crudAPI.Query<UserDocsClient>(debtor).Result;
//Now we have an array of documents attached to the debtor, each including its metadata
//But notice that the _Data field is null. We don't have the contents of each file yet
//To save bandwidth and speed up performance, Uniconta only sends "light" objects by default
//To get the contents into the _Data field, we must use the Read method
foreach (var doc in docs)
{
    _result = _crudAPI.Read(doc).Result;
    //Now the _Data field will hold the contents of the file
    //and it can now be used to save a local file
}
//This technique can be used on many entities to get extra data
```

3.7.4. Update / StreamingManager

The Update function can update a single entity or a list of entities.

```
DebtorClient debtor = _crudAPI.LoadCache<DebtorClient>().Result.Get("0001");
if (debtor != null)
{
    debtor._Address1 = "Test adresse 1";
    _result = _crudAPI.Update(debtor).Result;
}
```

Multiple users / StreamingManager

Usually, this is fine, but what if your code is running for a very long time between getting the data, updating it, and sending it back to Uniconta? Another user might have edited the same entity.

To ensure that only the fields that you have changed, are updated, you can use the StreamingManager to make a copy of your entity before changing it. If you send that back with the Update method, then it will take care to only update the changed fields.

Notice that last update always wins if the same field was edited by several users, but if another user edited a field you did not, you will not overwrite those values.

```
DebtorClient debtor = _crudAPI.LoadCache<DebtorClient>().Result.Get("0001");
if (debtor != null)
{
    var orig = StreamingManager.Clone(debtor);
    //or
    //DebtorClient orig = (DebtorClient)StreamingManager.Clone(debtor);
    debtor._Address1 = "Test adresse 1";
    _result = _crudAPI.Update(debtor,orig).Result;
}
```

3.7.5. Delete

```
DebtorClient debtor = _crudAPI.LoadCache<DebtorClient>().Result.Get("0001");
if (debtor != null)
{
    _result = _crudAPI.Delete(debtor).Result;
}
```

The method DeleteNoResponse will just send the command to Uniconta and start deleting. It can be used if you want to delete a huge amount of data, and not wait for a response

4. Tips & Tricks

4.1. Fields on client and non-client entities

When working with Client-type entities, you will see two versions of most fields. One version starting with underscore and one without. The fields starting with underscore are the database fields, and the ones without are the client representations of the same fields.

When updating fields, I recommend that you always use the database fields. This way you will be sure that you do not save unintentional data.

The example below shows how you might unintentionally try to save “corrupt” data through a client field.

You need to update the storage field, but if you use the client field “Storage” as shown below, then the plugin will only work on Uniconta with language set to Danish. Do not use this approach:

```
DebtorOrderLineClient orderLine = new DebtorOrderLineClient();
string orderLineStorage = "Reserveret";
orderLine.Storage = orderLineStorage;
```

Instead use the database field “_Storage” and set it to the related enum, which is language independent:

```
StorageRegister orderlineStorage = StorageRegister.Register;
orderLine._Storage = orderlineStorage;
```

4.2. Enums and how to parse

Uniconta uses Enums for many of its base entities. Currencies, Countries, Storage codes, Units and many other base entities, are stored as Enums.

Sometimes you need to get the Enum value of a string. You can use a built-in C# function for this, as illustrated below. (imp.Unit is imported text from a csv file and item is an InvItemClient)

Enum.TryParse will return true if the value exists as an enum and the variable “unit” will hold the Enum value

```
if (Enum.TryParse<ItemUnit>(imp.Unit, out ItemUnit unit))
{
    item._Unit = unit;
}
else
{
    //Handle error
}
```

4.3. Master / Detail queries

You can take advantage of the CrudAPI's ability to get related data, by using Master/Detail queries. Here are a few examples:

```
DebtorClient debtor = _crudAPI.LoadCache<DebtorClient>().Result.Get("0001");
//Get all sales orders for a customer
var orders = _crudAPI.Query<DebtorOrderClient>(debtor).Result;

//Get all order lines from all orders for that customer
var orderlines = _crudAPI.Query<DebtorOrderLineClient>(debtor).Result;

//Get all order lines from a specific order
DebtorOrderClient order = _crudAPI.LoadCache<DebtorOrderClient>().Result.First();
var orderlines = _crudAPI.Query<DebtorOrderLineClient>(order).Result;

//This master detail relationship can be used both ways
//If you have an order line, then you can easily get any of its masters
DebtorOrderLineClient line = new DebtorOrderLineClient { OrderNumber = "189" };
DebtorOrderClient order = _crudAPI.Query<DebtorOrderClient>(line).Result.FirstOrDefault();

//But getting The debtor from an orderline will not work. You cannot go back two steps.
//Instead get it from the order. This will not work
DebtorClient debtor = _crudAPI.Query<DebtorClient>(line).Result.FirstOrDefault();
//This will work:
DebtorClient debtor = _crudAPI.Query<DebtorClient>(order).Result.FirstOrDefault();
```

4.4. Working with Documents and Files

4.4.1. Import

```
//How to import a local file to a customer in Uniconta
//Read the contents of a file and store in a byte array
var bytes = File.ReadAllBytes(@"C:\temp\testfile.jpg");

//Create a new document
UserDocsClient newDoc = new UserDocsClient();
newDoc.Created = DateTime.Now;
newDoc.DocumentType = FileextensionsTypes.JPG;
newDoc.Text = "Test file";
newDoc._Data = bytes;

//Connect the document to an entity
DebtorClient debtor = _crudAPI.LoadCache<DebtorClient>().Result.Get("0001");
newDoc.SetMaster(debtor);

//Insert the document in Uniconta
_result = _crudAPI.Insert(newDoc).Result;
//Now the file is attached to the debtor in Uniconta
```

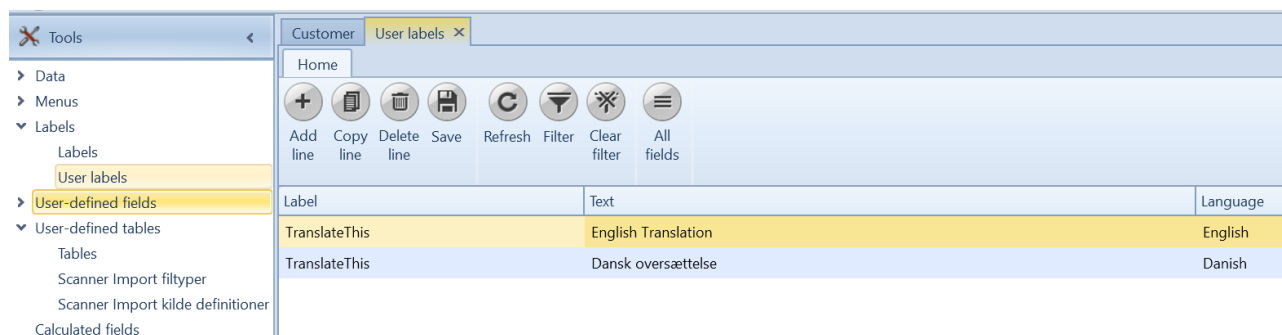
4.4.2. Export

```
//How to extract a file from Uniconta to a local file
//This sample takes the first file found on a customer
DebtorClient debtor = _crudAPI.LoadCache<DebtorClient>().Result.Get("0001");
var documents = _crudAPI.Query<UserDocsClient>(debtor).Result;
for (int i = 0; i < documents.Length; i++)
{
    //Read the file data
    _result = _crudAPI.Read(documents[i]).Result;
    var bytes = documents[i]._Data;
    File.WriteAllBytes($"C:\Temp\Testfile.{i}{documents[i]._DocumentType}", bytes);
}
```

4.5. Localization / Multi-language support

You can translate any custom text that you want, by using Uniconta labels.

First create the labels/translations for all the languages you want to support:



In your code, you can use the following method to get the translation of your label:

```
string translation = Uniconta.ClientTools.Localization.lookup("TranslateThis");
```

The above will use the language of the Uniconta client. If you want to force a specific language translation, then send the language as shown here:

```
string translation = Uniconta.ClientTools.Localization.lookup("TranslateThis", Language.da);
```

4.6. Performance tips

In general, the less calls you make to the Uniconta server, the better. Always think about the best way to structure the data you need for your application or plugin.

Try to get the master data you need only once, and reuse it across your plugin or application.

When developing plugins, use the cache whenever possible.

Gather the entities you need to send back to Uniconta in collections, and send the collection back in one call, instead of sending one entity at a time.

It is the same with querying data. It is advised to get all your entities in one call, instead of making separate calls for each single entity.

Of course, there are also exceptions, when it makes best sense to use the CrudAPI to send a query to the Uniconta database, asking for a single entity, and not an entire collection.

4.6.1. Caching

One way to avoid contacting the Uniconta server, when developing plugins, is through caching. Your plugins use the same cache, as the Uniconta client. There are both advantages and a drawbacks to using the cache.

How much you will use the cache depends on the type of plugin you develop and how many users work with the data you need at the same time.

An advantage to using the cache is fast access and no overhead, as no calls are made to Uniconta. The general rule is that it is recommended to use the cache for all access to Uniconta master data.

A drawback is that in a multi user environment, you may experience that the cache is not always updated to reflect the latest changes in the Uniconta database, on specific entities. If you are working with data, that is accessed by many users simultaneously, then you should test if there is any possible issue with the cache in your use case, and handle this in your code.

You must also think about the data you extract, and how long you keep that data in memory before accessing it. Is there a chance that the same data might have been updated by another user in the meantime?

Examples

To get a list of all orders from the existing cache:

```
List<DebtorOrderClient> orders = _crudAPI.LoadCache<DebtorOrderClient>().Result.ToList();
```

Get a list of all orders from the existing cache, but force an update of the cache, so you get all new orders created or modified, since the last cache update (Add the true parameter to the LoadCache call):

```
List<DebtorOrderClient> orders = _crudAPI.LoadCache<DebtorOrderClient>(true).Result.ToList();
```

Do not make it a habit to just always update the cache. Then you might as well just call the database directly:

```
List<DebtorOrderClient> orders = _crudAPI.Query<DebtorOrderClient>().Result.ToList();
```

4.6.2. Use C# functions to handle your data instead of the Uniconta server

The CrudAPI has many built in functions to ask the Uniconta server to sort, filter and package the data it sends back. I recommend that you use the cache, along with C# functions to do this instead. This is faster and no calls are made to the Uniconta server.

Sorting data

Sorting debtors by VAT number ascending:

```
List<DebtorClient> debtors =
_crudAPI.LoadCache<DebtorClient>().Result.OrderBy(x=>x.VatNumber).ToList();
```

Sorting debtors by VAT number descending

```
List<DebtorClient> debtors = _crudAPI.LoadCache<DebtorClient>().Result.OrderByDescending(x
=> x.VatNumber).ToList();
```

Filtering data

Only get the Customer with a specific VAT number

```
DebtorClient debtor = _crudAPI.LoadCache<DebtorClient>().Result.Where(x => x.VatNumber ==
"12345678").FirstOrDefault();
```

Only get customers in a specific group and Danish currency

```
List<DebtorClient> debtors = _crudAPI.LoadCache<DebtorClient>().Result.Where(x => x.Group ==
"Grp1" && x._Currency == Currencies.DKK).ToList();
```

4.6.3. Sending data back to Uniconta in collections

When you send data back to Uniconta, it is advised to group your data in collections of a size of 100 entities per call. This will optimize the performance of your application. If you have many thousand updates, then you may also group by 1000, but no more than 1000 entities per call is advised.

For example, if you have a big list of 12.000 Inventory items you want to create in Uniconta, then you can use this method to split your collection into a list of smaller lists, which can be sent with the Crud API:

```
public List<List<InvItemClient>> SplitList(List<InvItemClient> source, int listSize = 100)
{
    return source
        .Select((x, i) => new { Index = i, Value = x })
        .GroupBy(x => x.Index / listSize)
        .Select(x => x.Select(v => v.Value).ToList())
        .ToList();
}
```

This SplitList method will create a list of lists, each with up to 100 entities, which you can enumerate through. This is a sample of how to use the method (itemsToCreate is a List of InvItemClient):

```
var newSplitList = SplitList(itemsToCreate);
foreach (var list in newSplitList)
{
    _result = _api.Insert(list).Result;
    //Error handling here
}
```

Note: The SplitList method can be made generic, so it supports all Uniconta entities. This is demonstrated in a later chapter.

4.7. Get the right prices

Working with order lines through code can sometimes be a challenge. When you create a new order line through the Uniconta client, several fields are filled out automatically. When you create an order line through code, those same fields are not automatically filled out, and sometimes you have to mirror the logic from the Uniconta client in your plugin.

In the case of prices that can be very complex, but there is an API called DebtorCreditor, which has some tools to automate this task. The method below calculates the price on an orderline, where the debtor has a specific price list configured.

The code assumes that it only receives lines with Inventory items. Otherwise it will fail

```
private async Task CalcPriceLine(DebtorOrderLineClient line)
{
    var fPrice = new FindPrices(line.Order, _api);
    fPrice.UseCustomerPrices = true;
    await fPrice.loadPriceList();
    await fPrice.SetPriceFromItem(line,
        _api.LoadCache<InvItemClient>().Result.Get(line._Item));
    if (line.Price == 0)
    {
        //Use SalesPrice1 if no price was found
        line._Price = _api.LoadCache<InvItem>().Result.Get(line._Item)._SalesPrice1;
    }
}
```

The FindPrices functions are asynchronous, so the method should be called from an async method.

You can use the async version of IPluginBase, if you need to calculate prices like this.

Note: Remember that the line needs to have the basic properties like Item and Qty before the correct price is calculated

4.8. Make your Executable plugin run asynchronously

If you need to run your executable plugin asynchronously, then you can inherit from IPluginAsyncExecute. If you do this, then you will get an extra async method, where you can put your main plugin code. For example, this is an advantage when you need to calculate order line prices as described in chapter 4.7, or if your plugin needs to generate a report.

Below illustrates the difference from inheriting from IPluginBase versus IPluginAsyncExecute:

```
//The normal method is still here, but does nothing
ErrorCodes IPluginBase.Execute(UnicontaBaseEntity master, UnicontaBaseEntity currentRow,
IEnumerable<UnicontaBaseEntity> source, string command, string args)
{
    return ErrorCodes.Succes;
}

//This new async method is where you must put your code
public async Task<ErrorCodes> ExecuteAsync(UnicontaBaseEntity master,
UnicontaBaseEntity currentRow, IEnumerable<UnicontaBaseEntity> source, string command,
string args)
{
    //Main code here
}
```

4.9. Check for errors

Often the CrudAPI return value ErrorCodes.XXXX is not enough to identify the underlying issue. In those cases, you can get more details errors by accessing the session object, as shown below

```
StandardError[] error = _crudAPI.session.GetErrors().Result;
```

This error array now holds one or more errors, which you can investigate. In most cases you will get the info you need by looking at the message field.

This is a sample from inserting a project journal line, where the project does not exist. The result from CrudAPI is ErrorCodes.FieldHasInvalidValue, but by looking at the session error object we see which value causes the error:

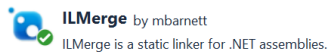
ErrorId	0
LineNo	1
Next	null
error	FieldHasInvalidValue
inFunction	"SQL Insert"
inProperty	null
message	"Unknown value '_Testprojekt' in Table: Project"
time	{24-08-2021 19:26:27}

4.10. The single DLL file – How to use ILMerge

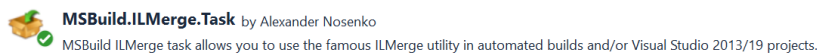
Uniconta only supports loading one DLL file as a partner plugin. This becomes a challenge when you use other NuGet packages, which come with their own DLL files. ILMerge comes to the rescue, as it will compile your project and all its references into one single DLL file.

4.10.1. How To: Use ILMerge with a Uniconta plugin:

1. Install the following two NuGet packages: (Do not worry that Visual Studio tells you that ILMerge is deprecated)



3.0.41

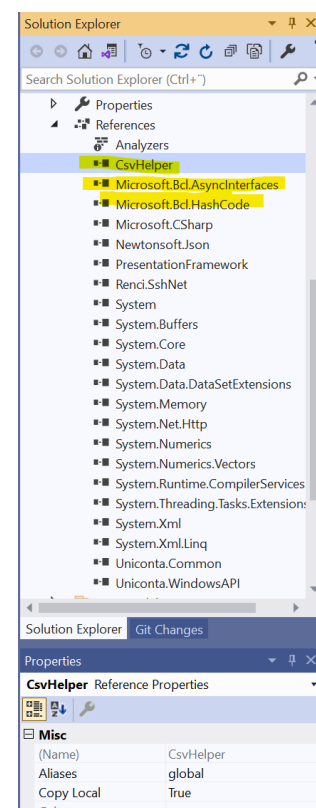
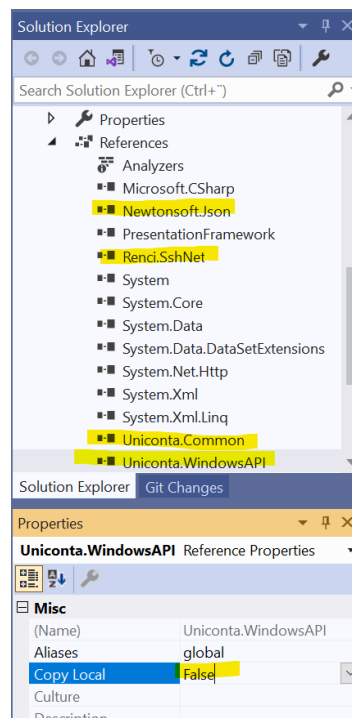


1.1.3

2. When the installation is finished, your project is automatically configured to compile all your references into a single DLL file. Now you need to exclude everything that is not necessary
3. In the Solution Explorer, unfold References, and go through each reference and check the “Copy To Local” value. If it is set to True, then that reference will be included in your Partner plugin DLL file. It is very important to only include the references that are absolutely necessary
4. You do not want to include the Uniconta NuGet packages, as they are already loaded by the Uniconta client

Go through all references and set to false. The four marked below are all part of the Uniconta package, so you want to set those to False. In this sample, the only external references I want to include is CSVHelper, so the following three will be set to true:

5. When you compile your project, you can see in the output window, that ILMerge is running, and merges all the references set to True into your finished DLL file



4.11. Refresh an overview page in Uniconta client

When you run an executable plugin and your code is finished updating data, you may want to refresh the overview in the Uniconta client. You can place the following three lines of code right before the return line:

```
PluginEventArgs arg = new PluginEventArgs();  
arg.EventType = PluginEvent.RefreshGrid;  
OnExecute(null, arg);
```

This will refresh the current overview page and make it reflect the changes your plugin has made.

4.12. Generics / Splitlist example

You want to write as little code as possible to get the job done. Generics will help you accomplish this. You can have one method supporting different object types, as long as they share the same interface. The Uniconta data model ensures this for all Uniconta entities. Therefore you can leverage Generics to accomplish more with less code. This example shows how to change the method SplitList used earlier, from a normal method to a generic method.

This is the normal method, where you would have to write a method for each entity you want to support:

```
public List<List<InvItemClient>> SplitList(List<InvItemClient> source, int listSize = 100)  
{  
    return source  
        .Select((x, i) => new { Index = i, Value = x })  
        .GroupBy(x => x.Index / listSize)  
        .Select(x => x.Select(v => v.Value).ToList())  
        .ToList();  
}
```

Below, the same method has been rewritten to a generic method, which supports all Uniconta entities. It can be called in the same way, as before:

```
public List<List<T>> SplitList<T>(List<T> source, int listSize = 100) where T : class,  
UnicontaBaseEntity, new()  
{  
    return source  
        .Select((x, i) => new { Index = i, Value = x })  
        .GroupBy(x => x.Index / listSize)  
        .Select(x => x.Select(v => v.Value).ToList())  
        .ToList();  
}
```


4.13. Find the active row when running an event-based plugin

Inheriting from `PageEventsBase`, you are given a page object, which refers to the current window in Uniconta. If this is an overview window (A Grid), then you can use the technique below to get the active row. This sample will work on the sales order overview.

When debugging, you will be able to see the text representation of a page, which you will use in your code.

```
if (page.ToString() == "UnicontaClient.Pages.DebtorOrders")
{
    UnicontaClient.Pages.DebtorOrders orderPage = page as UnicontaClient.Pages.DebtorOrders;
    UnicontaClient.Pages.DebtorOrdersGrid orderGrid = orderPage.gridControl as
UnicontaClient.Pages.DebtorOrdersGrid;
    DebtorOrderClient dbOrder = orderGrid.SelectedItem as DebtorOrderClient;
}
```

Now you have the marked order in the `dbOrder` object.

Be aware, that if you have made a customized version of the page, then the code will result in an error. You must add `CustomPage` to the namespace references as shown below:

```
if (page.ToString() == "UnicontaClient.Pages.CustomPage.DebtorOrders")
{
    UnicontaClient.Pages.CustomPage.DebtorOrders orderPage = page as
UnicontaClient.Pages.CustomPage.DebtorOrders;
    UnicontaClient.Pages.CustomPage.DebtorOrdersGrid orderGrid = orderPage.gridControl as
UnicontaClient.Pages.CustomPage.DebtorOrdersGrid;
    DebtorOrderClient dbOrder = orderGrid.SelectedItem as DebtorOrderClient;
}
```

4.14. StreamingManager

You can use the `StreamingManager` to convert between Uniconta entities and your own custom classes with user defined fields.

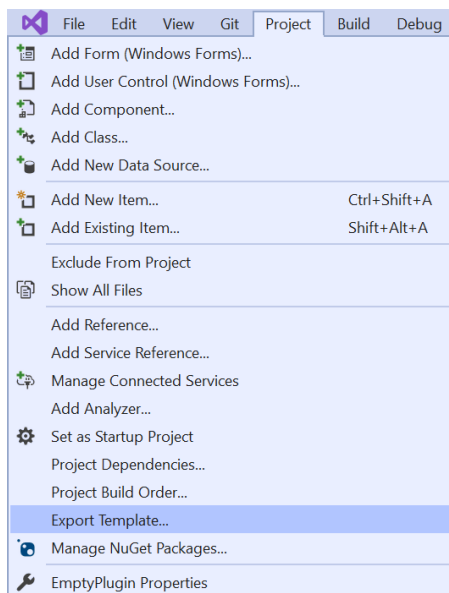
If you have an executable plugin running on an entity with user defined fields, and find that you cannot convert `currentRow` into your the custom class, then you can create a new empty object of the custom class, and use `StreamingManager.Copy` to copy `currentRow` into the custom class.

The Streaming Manager is also used when updating entities, to make sure that only the fields you have changed, are updated. If another user has changed some other fields, while your code is running, using the `StreamingManager`, can ensure that you do not overwrite the other user's data. See that sample in the chapter about working with the `CrudAPI`.

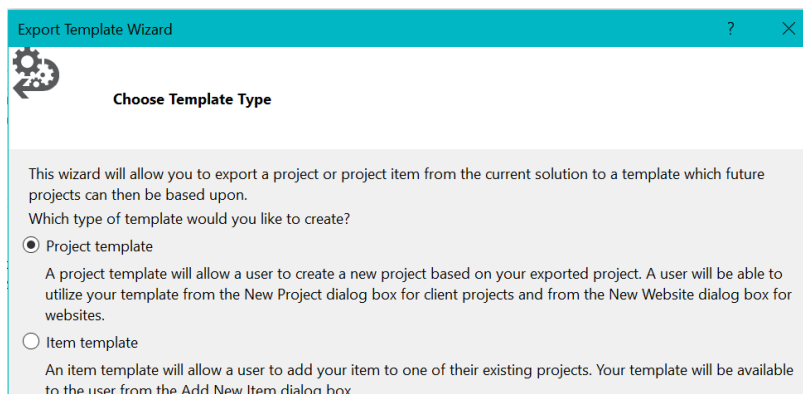
4.15. Create Visual Studio templates

You can create your own Visual Studio templates, in order to save time when creating new projects. I recommend creating a template for the most common project types you do.

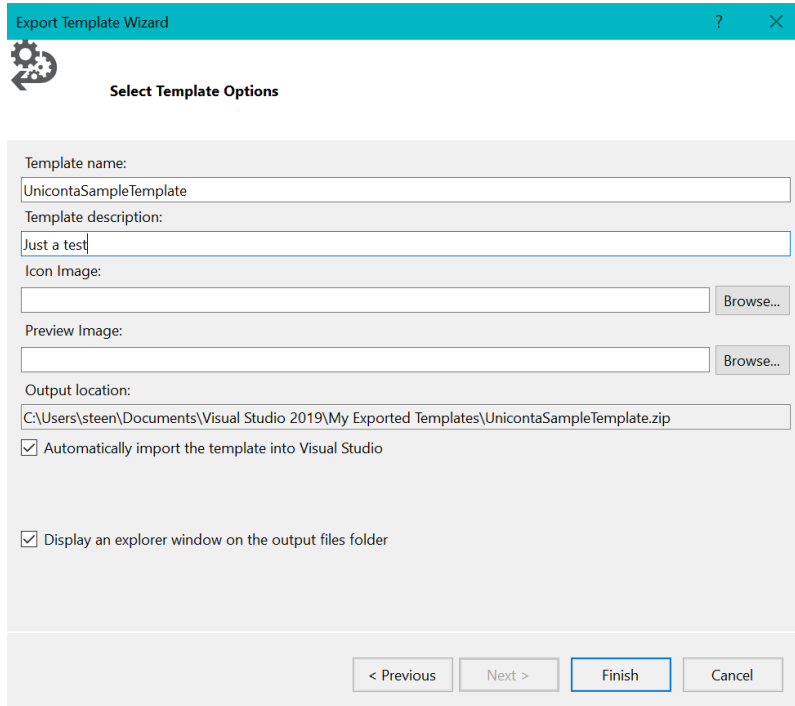
1. Create a new project and add any NuGet packages, folder and base code, which you will always need when creating this type of project
2. When you are happy with the base of your Visual Studio project, you can export it as a template by using the menu Project / Export Template



3. Choose Project Template



4. Give your template a name and a description



The image shows a screenshot of the 'Export Template Wizard' dialog box in Visual Studio. The title bar is teal with a question mark and a close button. Below the title bar is a gear icon and the text 'Select Template Options'. The dialog contains several input fields and checkboxes. The 'Template name' field is filled with 'UnicontaSampleTemplate'. The 'Template description' field is filled with 'Just a test'. The 'Icon Image' and 'Preview Image' fields are empty, each with a 'Browse...' button to its right. The 'Output location' field is filled with 'C:\Users\steen\Documents\Visual Studio 2019\My Exported Templates\UnicontaSampleTemplate.zip'. There are two checkboxes: 'Automatically import the template into Visual Studio' (checked) and 'Display an explorer window on the output files folder' (checked). At the bottom, there are four buttons: '< Previous', 'Next >', 'Finish' (highlighted with a blue border), and 'Cancel'.

Export Template Wizard

Select Template Options

Template name:
UnicontaSampleTemplate

Template description:
Just a test

Icon Image:
Browse...

Preview Image:
Browse...

Output location:
C:\Users\steen\Documents\Visual Studio 2019\My Exported Templates\UnicontaSampleTemplate.zip

☒ Automatically import the template into Visual Studio

☒ Display an explorer window on the output files folder

< Previous Next > Finish Cancel

5. When you click Finish, the template is automatically imported and made available the next time you start the project wizard. You also get a ZIP file, which you can backup, and keep if you need to reinstall your PC in the future.
6. If you later need to restore the template on a fresh installation, then copy the ZIP file to the following folder on your new machine:
%USERPROFILE%\Documents\Visual Studio 2019\Templates\ProjectTemplates

5. Links

Uniconta Github site

<https://github.com/Uniconta>

Request a new API key

<https://www.uniconta.com/developers/uniconta-api-key-application/>

Uniconta client download

<https://www.uniconta.com/downloads/>

Downloads for developers

<https://www.uniconta.com/developers/download-for-developers/>

Explanation of differences between .NetStandard 2.0 and 2.1:

<https://docs.microsoft.com/en-us/dotnet/standard/net-standard>

Uniconta Icon library

<https://www.uniconta.com/da/plugins/icon-dictionary/>

DevExpress subscriptions:

<https://www.devexpress.com/buy/net/>

6. Exercises

Exercise files will be sent to course participants, and solutions will be provided.

We will go through working solutions to all exercises during the course, where we will also discuss alternative options.

6.1. Exercise 1: Create Visual Studio Uniconta template

In this exercise, you must create a Visual Studio Template for a Uniconta plugin, which you can use for your future projects. This template will save you time when creating new Uniconta plugins.

1. Follow the guide in chapter 2.3.2 on how to create your first executable plugin
2. Follow the guide in Chapter 4.15 on how to create a Visual Studio Template

6.2. Exercise 2: Event plugin and user-defined fields - Calculating sales prices based on coverage

A customer wants to have three new fields on his Inventory Items, one for each Sales price field. Each field will be used to calculate the related sales price, and should be named Coverage 1, 2 and 3. A percentage (Integer) will be typed into the coverage fields, and the following calculation must be used to calculate the resulting salesprice: $\text{SalesPriceX} = \text{CostPrice} / (1 - (\text{CoveragePct} / 100))$

The customer wants to have the related salesprice calculated, as soon as the coverage is typed in, and he tabs to the next field.

If the Cost price is changed on an item, all three salesprices must be recalculated, if the Coverage fields have a value.

The Coverage fields must be created as User defined fields.

Extra challenge

The Customer wants to always round up to the nearest 0.5. So, if the sales price is 2.34, the result must be 2.5 and if the sales price is 2.54, the result must be 3.00.

Hints:

You need to override the method "Record_PropertyChanged"

To check which property has been changed in the Uniconta client, you can check the value of `e.PropertyName` in the following way: `if (e.PropertyName == "fldCoverage1")`

Use this call to tell the Uniconta client, that you have updated the value of a field. This will refresh the field and update the field value in the Uniconta client (item is the current open In: `item.NotifyPropertyChanged("NameOfField");`

Steps to complete this exercise

1. Create user defined fields on Inventory Items
2. Create an event based plugin
3. Create a method that overrides "Record_PropertyChanged"
4. Write your custom code to check which property was changed, and update as needed
5. Compile your plugin to your local pluginpath. By default `c:\uniconta\pluginpath`
6. Create a user plugin in Uniconta
7. Test and debug your plugin

6.3. Exercise 3: Executable plugin - Import a CSV file with Inventory items

A customer wants to import a list of Inventory items from a CSV file. If the Inventory item exists in Uniconta, then its data must be updated. If the item doesn't exist, then it must be created.

The CSV file has the following fields

ItemNo;Name;Type;CostPrice;SalesPrice;Unit;Group;ImportantInfo

The field "ImportantInfo" is not a standard Uniconta field, and must be created as a user-defined field.

Hints

You can either import the custom InventoryItem class to your code, or refer to the user defined field by its index

To read and convert the CSV file, you have many options. Two options are either to use the NuGet package CsvHelper along with ILMerge, or you can use the class TextFieldParser, which is included in the namespace Microsoft.VisualBasic.FileIO;

If you go with TextFieldParser, here is some code to begin with.

Use the class with using statement, and then configure delimiter and other properties

```
using (TextFieldParser csvParser = new TextFieldParser(path, Encoding.UTF8))
{
    //Code to configure csvParser and read lines from file here
    csvParser.SetDelimiters(new string[] { ";" });

    while (!csvParser.EndOfData)
    {
        //To read the next line from the file you can use
        string[] fields = csvParser.ReadFields();

        //To read a specific column. In this case column 3:
        string fieldname = ReadField(fields, 2);

        //Get all data from the file
    }
    //Save data in Uniconta
}
```

Example of a ReadField method:

```
private string ReadField(string[] fields, int location)
{
    return fields[location].ToString().Trim();
}
```

If you want to ask a user for a file, then you can use this method:

```
private string AskForFile()
{
    OpenFileDialog fdg = new OpenFileDialog
    {
        CheckFileExists = true,
        CheckPathExists = true,
        Title = "Vælg import fil",
        Multiselect = false
    };
    if (fdg.ShowDialog() == true)
    {
        string path = fdg.FileName;
        if (File.Exists(path))
        {
            return path;
        }
    }
    return string.Empty;
}
```

Steps to complete this exercise

1. Create user defined field on Inventory Items
2. Create an executable plugin
3. Decide how you want to read the CSV file
4. Write your custom code to read the CSV file and update or create items as needed
5. Compile your plugin to your local pluginpath. By default c:\uniconta\pluginpath
6. Create a user plugin in Uniconta
7. Test and debug your plugin

6.4. Exercise 4: Event plugin - Export a CSV file with invoice details when a sales order is invoiced

A customer wants to export a csv file with order details, each time a sales order is invoiced.

The file name must be equal to the invoice number. If the Invoice number is 1234, then the file must be named 1234.csv

The following fields must be in the file, and each order line with Qty > 0 must have a line in the file:

OrderNo;Item;Qty;Price;CustomerAccount

Hints

As in the previous exercise, you can use the NuGet package CsvHelper along with ILMerge or use the built-in functions of C# to generate the CSV file.

If you go for the second option, then you can utilize the StringBuilder class to create the contents of the CSV file, and the following method to write it:

For small files, you can also use the following method to write a line to a file

```
public void WriteToFile(string input)
{
    using (FileStream fileStream = new FileStream(_outputfilepath, FileMode.Append,
        FileAccess.Write))
    {
        using (var outputFile = new StreamWriter(fileStream))
        {
            outputFile.WriteLine(input);
        }
    }
}
```

You can catch the invoicing by an event based plugin, and looking at the right ActionTypes PreCreateInvoice and PostCreateInvoice.

If a use clicks cancel to the invoice dialog, then it is not clear to your plugin. One way to check if a user has cancelled the invoicing process is to count the number of invoices before and after the dialog, and compare these numbers. Another way is to compare the next number in the invoice number series.

Steps to complete this exercise

1. Create an event based plugin
2. Create a method that overrides "OnMenuItemClicked"
3. Write your custom code to handle both Pre- and Post events when a user generates an invoice
4. Compile your plugin to your local pluginpath. By default c:\uniconta\pluginpath
5. Create a user plugin in Uniconta. The plugin should work on both sales order overview and orderlines overview
6. Test and debug your plugin with a sales order

6.5. Exercise 5: External console application – Invoice a list of sales orders from a file

A customer wants a console application, which can be used to read a file with a list of order numbers, which must be invoiced in Uniconta. The file has no header, only a list of order numbers with one order number on each line.

Steps to complete this exercise

1. Create a Console application project
2. Write the code that connects to Uniconta
3. You can use the same technique as in Exercise 3 to read values from the file
4. You can call the method `PostInvoice` in the `InvoiceAPI` to generate an invoice
5. Run the application and verify that orders have been invoiced

6.6. Exercise 6: Modify a Uniconta page to have custom logic

A customer wants an extra field on the purchase order lines overview. The field must show extra detail about the item on the line. The data must be taken from the Item name group, which is registered on the Vendor of the order.

The new field must show both the Customer item number and the description from the Item name group, with a space between.

The data must be updated every time order lines are opened, and they must remain when the user refreshes the order lines.

Hints

You can add the column by creating a user defined field on purchase order lines.

You can subscribe to the Loaded event, in order to catch when Uniconta is finished filling out the standard details of the overview. This subscription can be registered in the Init method of the standard code, with the following line: `dgCreditorOrderLineGrid.Loaded += NameOfYourMethod;`

You also need to find the place, where the Refresh action is located in the standard code, and call your method from there too.

Steps to complete this exercise:

1. Create a user defined field on Creditor orderline
2. Create a Uniconta page-based plugin project, as described in chapter 3.4.3 (Only the first 4 steps)
3. Add the following files to the Pages folder (Download from Uniconta's GitHub):

CreditorOrderLines.xaml
CreditorOrderLines.xaml.cs

4. Edit the file CreditorOrderLines.xaml.cs and add your custom code.
5. If you are not using the project supplied by Uniconta, then add references to PresentationCore, WindowsBase, and System.xaml
6. Create a user plugin in Uniconta of type Page
7. Compile and test the plugin

6.7. Exercise 7: Event Plugin – Block invoicing

A customer wants a plugin, which blocks invoicing of an order, if the order has an order group set to the group “NotInvoicable”.

The plugin must show a message box to the user, that invoicing is not allowed on this order, and Uniconta must never show the invoicing dialog.

If the order is not part of the order group, then Uniconta must work as normal.

Steps to complete this exercise:

1. Create the order group if it doesn't already exist
2. Create a sales order, which is part of this order group, or change the order group on an existing sales order
3. Create an event based plugin
4. Create a method that overrides “OnMenuItemClicked”
5. Write your custom code to check the order group and block invoicing.
6. Create a user plugin and test your code

6.8. Exercise 8: Executable Plugin – Set field on order

A customer wants a function on the sales order overview, which sets the field “OurRef” on the marked order, to the ID of the logged in user. The function must be activated from the User menu and must be called “Take order”.

The plugin must only work on the marked order.

Hints:

- You can convert `currentRow` to a `DebtorOrderClient` with this code:
`DebtorOrderClient order = currentRow as DebtorOrderClient`
- Then you can set the value and save the order.
- Remember to update the grid.

Steps to complete this exercise:

1. Create an executable plugin
2. Write your custom code
3. Create a user plugin and test your code